



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2023/2024

Fundamentos de Sistemas Operativos
Relatório: Trabalho Prático 2 - Aula 6

Turma: 31D	Grupo: 4
Nome: João Ramos	Número: 50730
Nome: Miguel Alcobia	Número: 50746
Nome: Fábio Pestana	Número: 50756
Professor: Jorge Pais	

Data: 25 de Dezembro de 2023

Índice

Lista de figuras.....	III
Abreviaturas e símbolos.....	IV
Objetivos.....	1
Aula 1 - Desenho das GUI's das tarefas e do processo REI_SUBDITO.....	2
1. 1 - Desenho da GUI REI_SUBDITO.....	2
1.2 - Desenho da GUI_REI e GUI_SUBDITO.....	2
1. 3 - Desenho da GUI_GRAVAR.....	3
1. 4 - Alterações e última versão.....	3
Aula 2 - Desenho do diagrama de atividades do REI e respetiva implementação.....	6
1. 1 - Desenho do diagrama de atividades do REI.....	6
1.2 - Implementação do comportamento REI.....	7
Aula 3 - Desenho do diagrama de atividades do SUBDITO e respetiva implementação.....	7
1. 1 - Desenho do diagrama de atividades do Súdito.....	8
1.2 - Implementação do comportamento Súdito.....	9
Aula 4 - Desenho do diagrama de atividades do Gravar e respetiva implementação.....	10
1. 1 - Desenho do diagrama de atividades do Gravar.....	10
1.2 - Implementação do comportamento Gravar.....	10
Aula 5 - Teste de Implementação para controlo do Rei_Subdito.....	12
1. 1 - Implementação para lançar as tarefas pelo Rei_Subdito.....	12
Aula 6 - Teste da aplicação multitarefa completa.	14
1. 1 - Teste da aplicação.....	14
Conclusão.....	15
Bibliografia.....	16
Código.....	17
Aula 6 - Teste da aplicação multitarefa completa:.....	17
Código GUI REI_SUBDITO:.....	17
Código GUI Gravar:.....	20
Código Classe MyRobotLegoEV3:.....	23
Código Classe Gravar_Spy:.....	24
Código Classe Gui Genérica:.....	29
Código Classe Gui Rei:.....	33
Código Classe Gui Súdito:.....	34
Código Classe Rei:.....	36

Código Classe Subdito:	40
Código Classe BufferCircular:	43
Código Classe Mensagem:	45
Código Classe MensagemTempo:	46
Código Classe CriarMensagem:	46
Código Classe Dados:	48
Código Interface Gravar_Spy:	48
Código Interface Rei:	49
Código Interface Subdito:	49
Código Interface Mensagem:	50

Lista de figuras

Figura 1 - GUI feita para o REI_SUBDITO	2
Figura 2 - GUI's do Rei e do Súbdito.....	3
Figura 3 - GUI Gravar.....	3
Figura 4 - Nova GUI do REI_SUBDITO	4
Figura 5 - Nova GUI do Subdito	4
Figura 6 - Nova GUI do Rei.....	5
Figura 7 - Nova GUI do Gravar.....	5
Figura 8 - Diagrama do autômato principal do REI.....	6
Figura 9 - Diagrama do autômato Funcionar.....	6
Figura 10 - Diagrama do autômato Comunicação do Súdito.....	8
Figura 11 - Diagrama do autômato exeComando_Robot.....	8
Figura 12 - Diagrama do autômato Comunicação do Gravar	10
Figura 13 - Diagrama de Classes do TP2.....	17

Abreviaturas e símbolos

Lista de abreviaturas

GUI	Graphical User Interface
API	Application Programming Interface

Objetivos

Neste trabalho, os alunos terão como objetivo desenvolver um processo REI_SUBDITO, constituído por três tarefas: uma que implementa o comportamento REI; outra o comportamento SUBDITO, que conhece a API do robot, e a terceira tarefa é a GRAVAR que permite gravar alguns movimentos do robot num ficheiro.

Este trabalho é dividido em 6 partes distribuídas pelas seguintes aulas:

1ª aula prática – Desenvolvimento das GUI's das três tarefas e do processo REI_SUBDITO;

2ª aula prática – Desenho do diagrama de atividades do comportamento REI. Implementação e teste deste comportamento individualmente;

3ª aula prática – Desenho do diagrama de atividades do comportamento SUBDITO. Implementação e teste deste comportamento conjuntamente com o comportamento REI e com o movimento do robot.

4ª aula prática – Desenho do diagrama de atividades da tarefa GRAVAR. Implementação e teste desta tarefa conjuntamente com a tarefa SUBDITO e com o movimento do robot.

5ª aula prática – Implementação e teste do lançamento e sincronização entre o processo REI_SUBDITO e as tarefas REI, SUBDITO e GRAVAR de modo ao processo REI_SUBDITO conseguir ativar ou desativar os diferentes comportamentos.

6ª aula prática – Teste da aplicação multitarefa completa. Demonstração ao Professor e respetiva validação.

Aula 1 - Desenho das GUI's das tarefas e do processo REI_SUBDITO

1.1 - Desenho da GUI REI_SUBDITO

Seguindo as indicações do enunciado:

“O processo REI_SUBDITO, bem como cada tarefa que o constitui, têm associados uma GUI com uma consola para debugging, cujo desenho é deixado à criatividade de cada grupo de alunos.”

Elaborou-se a seguinte GUI com o auxílio do editor gráfico WindowBuilder no Eclipse:

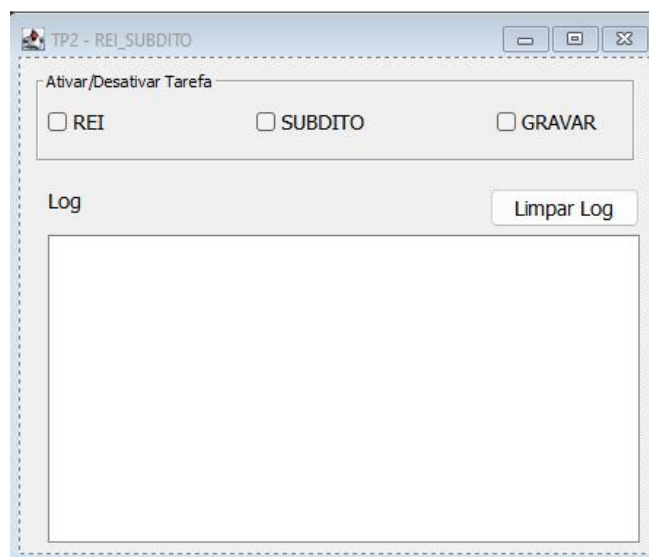


Figura 1 - GUI feita para o REI_SUBDITO

1.2 - Desenho da GUI_REI e GUI_SUBDITO

Para as GUI's do Rei e do Súdito foram aproveitadas as GUI's desenvolvidas no TP1, durante a aula 1. Contudo, o campo do “Canal de Comunicação”, onde o ficheiro para o canal era escolhido, foi retirado.

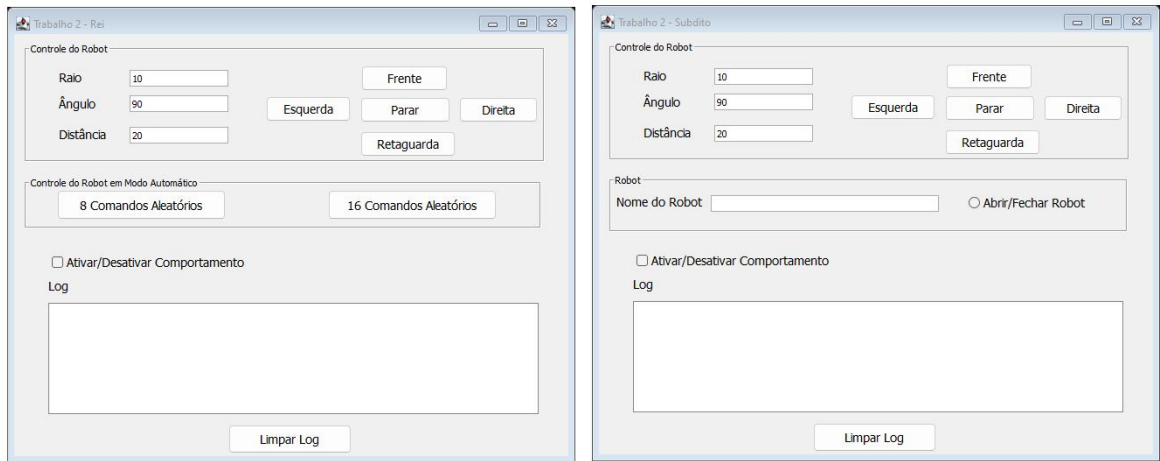


Figura 2 - GUI's do Rei e do Súdito

1. 3 - Desenho da GUI_GRAVAR

O comportamento GRAVAR tem na sua GUI três opções: “Gravar/Parar”, “Reproduzir”, “Definir o nome do ficheiro”.

Em “Gravar/Parar”, é permitido gravar ou parar a gravação num ficheiro, alguns movimentos enviados para o robot, para serem executados mais tarde.

Em “Reproduzir”, é permitido reproduzir os movimentos gravados num ficheiro e, durante uma reprodução não é permitido que os outros comportamentos enviem comandos para o robot.

Em “Definir o nome do ficheiro”, defini-se o nome do ficheiro para gravação ou reprodução.

Elaborou-se a seguinte GUI com o auxílio do editor gráfico WindowBuilder no Eclipse:

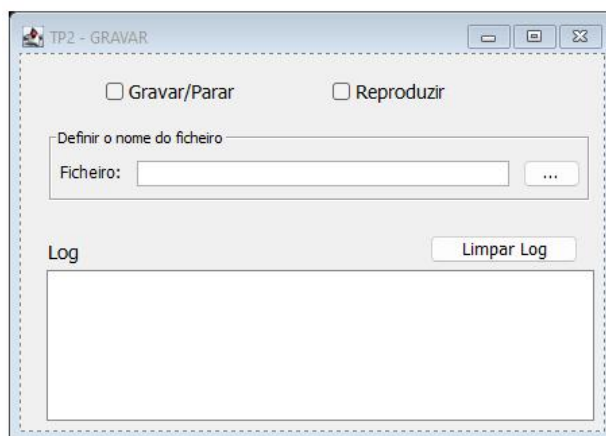


Figura 3 - GUI Gravar

1. 4 - Alterações e última versão

No decorrer do trabalho, o grupo decidiu trocar alguns campos das GUI's de lugar para a tornar mais intuitiva, como trocar as *checkboxes* da GUI do GRAVAR. Além disso, também se optou por

fazer uma nova paleta de cores para as GUI's, num estilo de “modo escuro” (*dark mode*).

Eis os resultados:

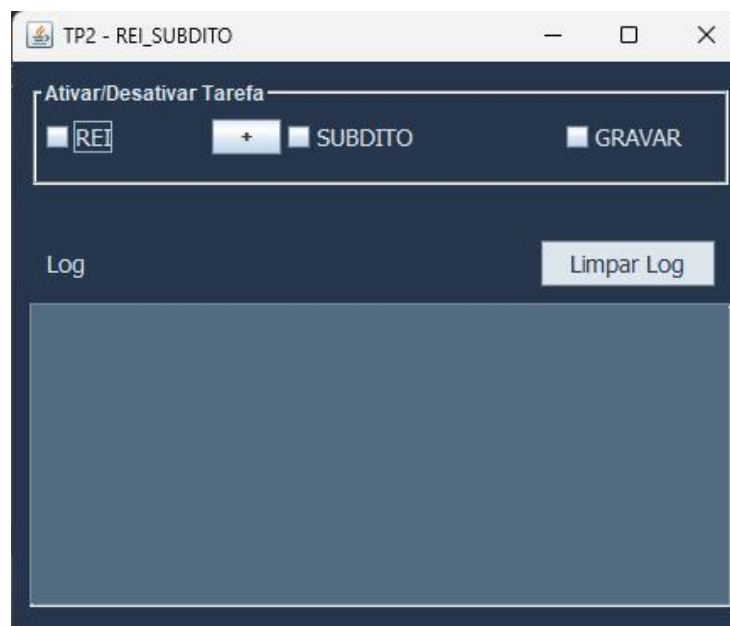


Figura 4 - Nova GUI do REI_SUBDITO

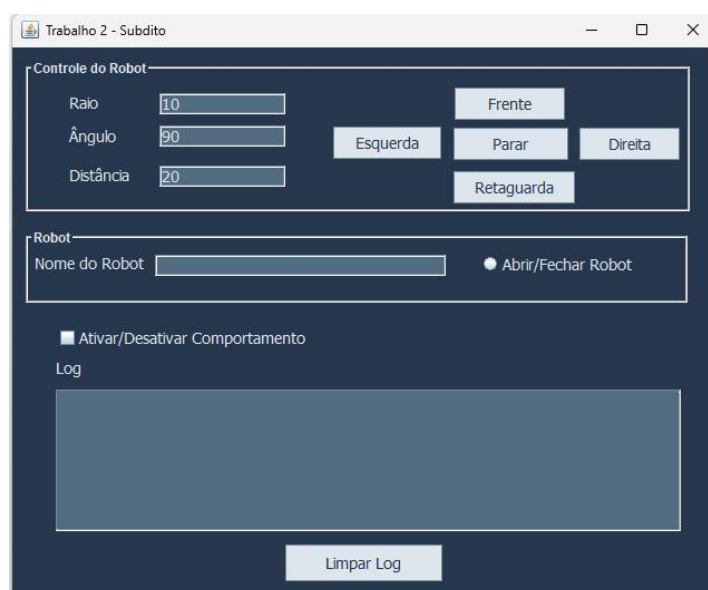


Figura 5 - Nova GUI do Subdito

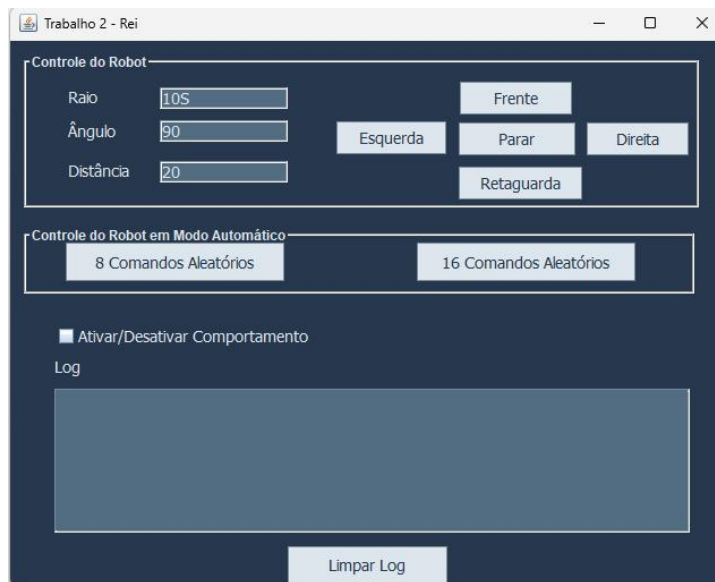


Figura 6 - Nova GUI do Rei

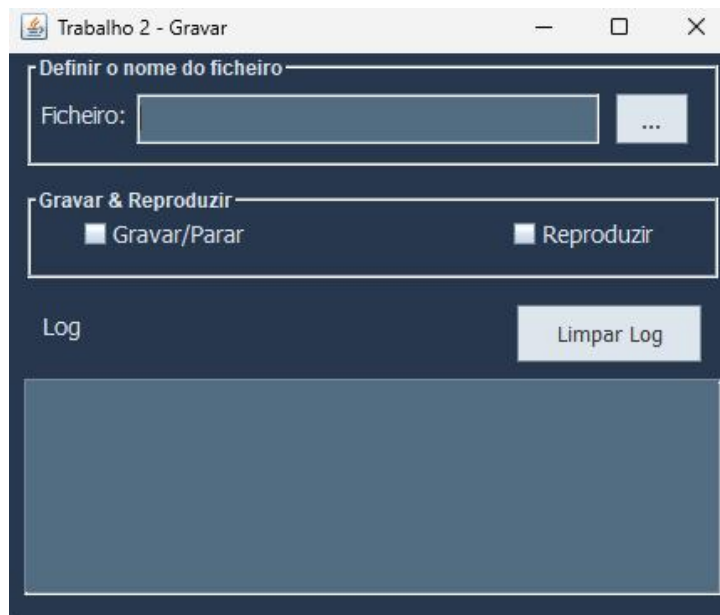


Figura 7 - Nova GUI do Gravar

Aula 2 - Desenho do diagrama de atividades do REI e respetiva implementação

1. 1 - Desenho do diagrama de atividades do REI

Conseguiu-se aproveitar alguma da estrutura e da lógica desenvolvida no TP1 e com as devidas alterações e correções, chegou-se ao seguinte diagrama:

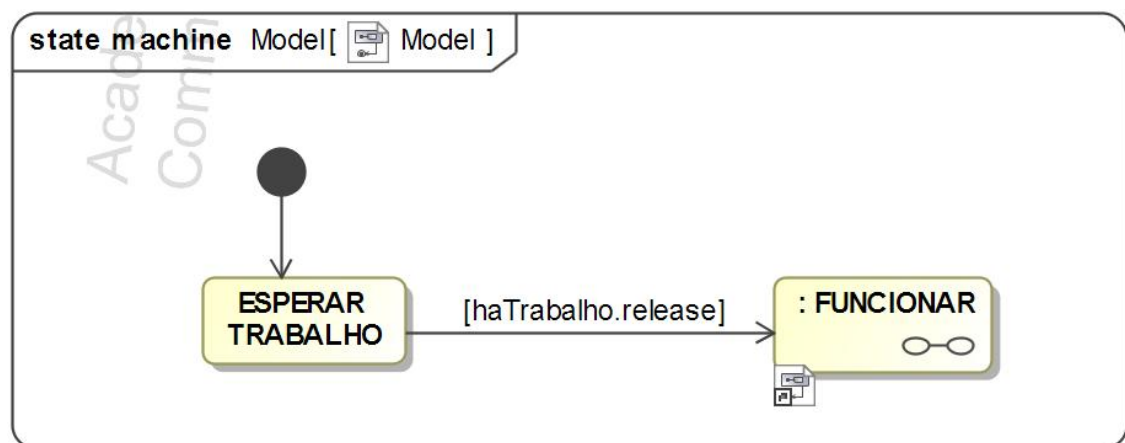


Figura 8 - Diagrama do autómato principal do REI

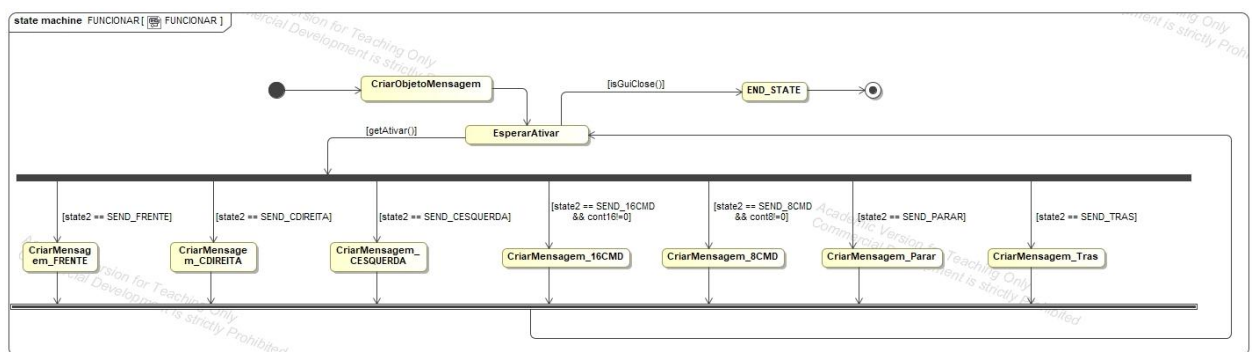


Figura 9 - Diagrama do autómato Funcionar

O primeiro diagrama mostra dois estados: `ESPERAR_TRABALHO` e `FUNCIONAR`. No primeiro estado, faz-se *acquire* do semáforo `haTrabalho` e se for possível muda-se para o estado `FUNCIONAR`, onde arranca um segundo autómato.

Este autómato começa no estado `CRIAR_OBJETO_MENSAGEM` e lá cria-se um objeto do tipo `CriarMensagem` para ser usado ao longo do processo. Depois, no estado `ESPERAR_ATIVAR`, consoante o botão seleccionado na GUI do Rei muda-se para o estado de criação da respetiva mensagem, isto se a *checkbox* de ativar e desativar comportamento estiver ativada. O controlo do funcionamento é feito através do semáforo `podeFuncionar` que recebe permissões, quando se clica nos botões.

Em seguida, está um exemplo do tratamento de eventos no botão `FrenteButton`, o mesmo tratamento foi realizado para as restantes direções:

```
getFrenteButton().addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent arg0) {  
        rei.getPodeFuncionar().release();  
        rei.setState2(IRei.SEND_FRENTE);  
    }  
});
```

1.2 - Implementação do comportamento REI

Para a implementação seguiu-se o mesmo modelo do TP1, onde se aplicou a estrutura do diagrama de atividades num `switch case`. No caso, cada estado do diagrama corresponde a uma `case` do `switch`.

Para escrever e ler as mensagens foi desenvolvida uma nova classe `BufferCircular`, que tem por base a utilização de semáforos.

A classe `Buffer circular` tem como atributos:

- `buffer`: Um array de Mensagens que cumpre o papel de `buffer`.
- `put`: Um índice que marca a posição de escrita no `buffer`.
- `get`: Um índice que marca a posição de leitura no `buffer`.
- `size`: O tamanho do `buffer`.
- `elementosLivres`: Controla o número de espaços disponíveis no `buffer` para escrita.
- `elementosOcupados`: Controla o número de elementos ocupados no `buffer`, prontos para leitura.
- `acessoElementos`: Garante acesso por exclusão mútua durante a leitura e a escrita no `buffer`.

Esta classe ainda conta com dois métodos:

- `ler()`: responsável por ler uma mensagem do `buffer`. O semáforo `elementosOcupados`, através do seu `acquire()`, garante que existem mensagens disponíveis para leitura antes de continuar. Em seguida, obtém acesso ao `buffer` pelo `acessoElementos.acquire()`, lê a mensagem na posição pretendida, atualiza o índice de `get` e liberta os semáforos `acessoElementos` e `elementosLivres`.
- `Escrever()`: responsável por adicionar uma mensagem ao `buffer`. O semáforo `elementosLivres.acquire()` garante que existe espaço disponível no `buffer` antes de continuar. Em seguida, obtém acesso ao `buffer` (`acessoElementos.acquire()`) e escreve a mensagem na posição pretendida, atualiza o índice de `put`. Por fim, liberta os semáforos `acessoElementos` e `elementosOcupados`.

Aula 3 - Desenho do diagrama de atividades do SUBDITO e respetiva implementação

1.1 - Desenho do diagrama de atividades do Súdito

Conseguiu-se aproveitar alguma da estrutura e da lógica desenvolvida no TP1 e com as devidas alterações e correções, chegou-se ao seguinte diagrama:

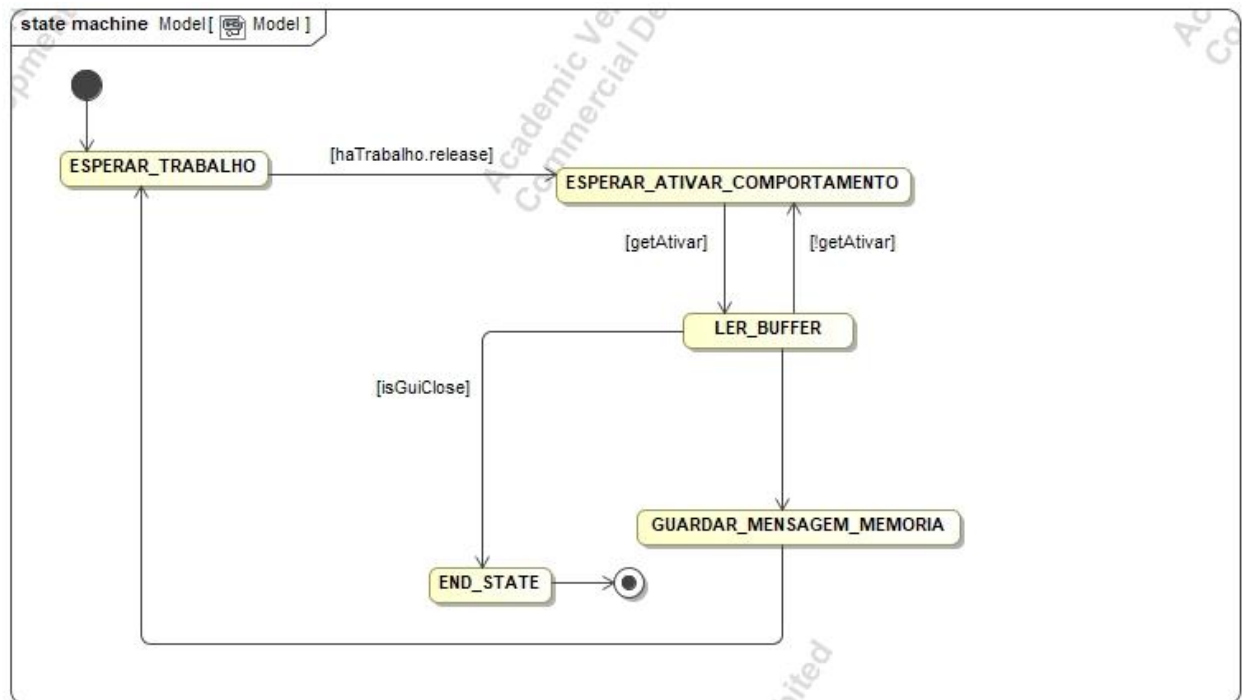


Figura 10 - Diagrama do autômato Comunicação do Súdito

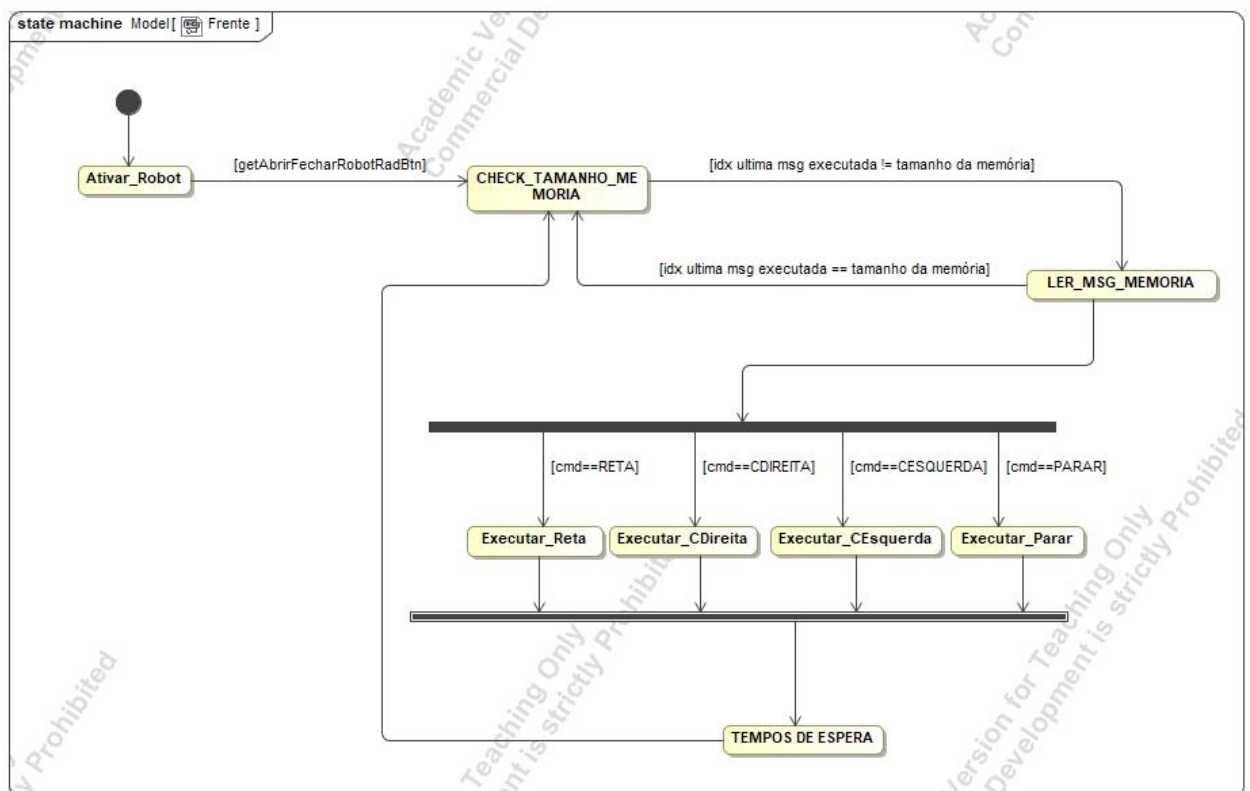


Figura 11 - Diagrama do autômato exeComando_Robot

O primeiro diagrama começa no estado `ESPERAR_TRABALHO`, onde se faz *acquire* do semáforo `haTrabalho` e se for possível muda-se para o estado `ESPERAR_ATIVAR_COMPORTAMENTO`, onde muda para o estado `LER_BUFFER` caso a *checkbox* “Ativar/Desativar comportamento” esteja marcada.

No `LER_BUFFER` o buffer é lido e passamos para o estado `GUARDAR_MENSAGEM_MEMORIA`. Se a GUI for fechada, passamos para o `END_STATE`.

O segundo autômato começa no estado `ATIVAR_ROBOT` e lá verificasse se a ligação com robot foi efetuada com sucesso. Se o índice da última mensagem for diferente do tamanho da memória, a leitura da memória é efetuada. Depois, consoante o número de `cmd` que a Mensagem tem, muda-se para o estado de `Executar_X`, onde X é o comando que corresponde a `cmd`. Por último, esperasse o tempo necessário para a mensagem ser executada no robot.

1.2 - Implementação do comportamento Súbdito

Para a implementação seguiu-se o mesmo modelo do TP1, onde se aplicou a estrutura do diagrama de atividades num `switch case`. No caso, cada estado do diagrama corresponde a uma `case` do `switch`.

O *releases* do `haTrabalho` é feito no estado `GUARDAR_MENSAGEM_MEMORIA`, mesmo antes de voltarmos para o `ESPERAR_TRABALHO`.

Aula 4 - Desenho do diagrama de atividades do Gravar e respectiva implementação

1.1 - Desenho do diagrama de atividades do Gravar

Seguindo as indicações do que é suposto implementar neste comportamento, chegou-se ao seguinte diagrama:

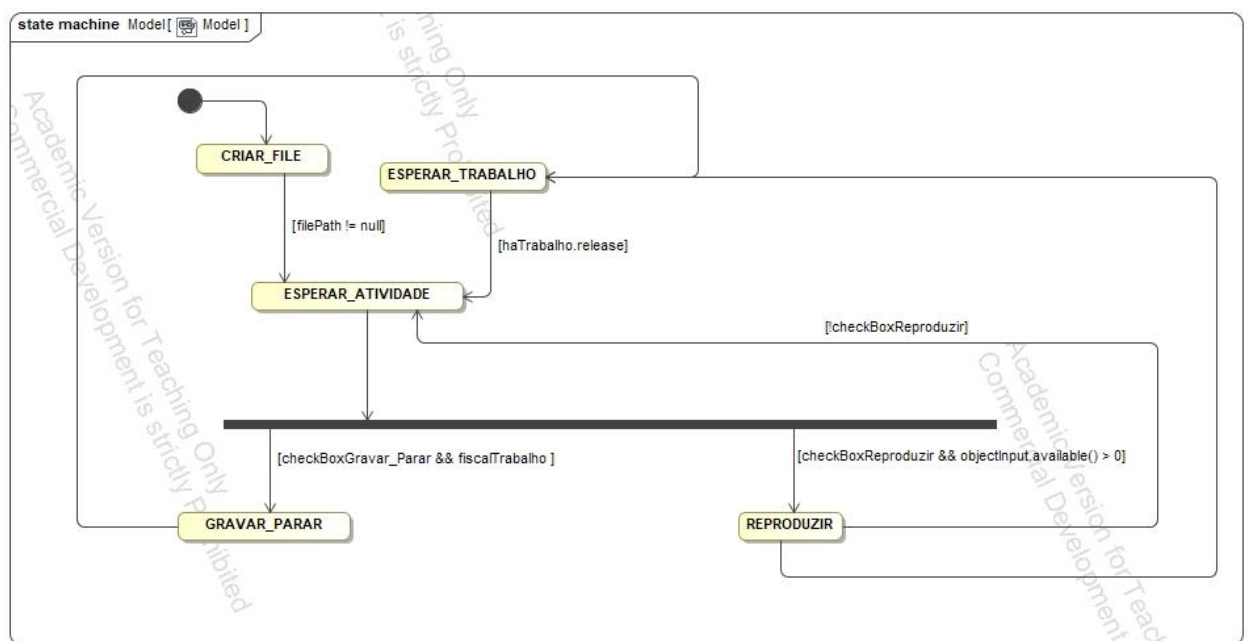


Figura 12 - Diagrama do autômato Comunicação do Gravar

O diagrama começa no estado `CRIAR_FILE`, onde criamos o ficheiro que será usado para a gravação dos comandos, para a sua leitura e para uma futura reprodução. Muda-se para o estado `ESPERAR_ATIVIDADE`, onde muda para o estado `GRAVAR_PARAR`, caso a *checkbox* “Gravar” esteja marcada e os comandos forem executados (que fazem com que o booleano `fiscalTrabalho` passe a `true`). Também pode ir para o estado `REPRODUZIR`, caso a *checkbox* “Reproduzir” esteja marcada e o ficheiro ainda tenha informação por ler.

Se as *checkboxes* forem desmarcadas, volta-se para o estado `ESPERAR_ATIVIDADE`. Contudo, se continuarem ativas, entra-se no estado `ESPERAR_TRABALHO`, onde se espera a disponibilidade do Semáforo `haTrabalho` para prosseguir.

1.2 - Implementação do comportamento Gravar

No `GRAVAR_PARAR` usa-se o `ObjectOutputStream.write()` assim como o `ObjectOutputStream.flush()` para escrever as Mensagens no ficheiro; enquanto no `REPRODUZIR` usa-se o `ObjectInputStream.read()` para ler a informação do ficheiro.

No `ESPERAR_TRABALHO` é feito o *acquire* do Semáforo `haTrabalho`, e o seu *release* é realizado após cada passagem pelo estado `GRAVAR_PARAR` e `REPRODUZIR`.

Aula 5 - Teste de Implementação para controlo do Rei_Subdito

1. 1 - Implementação para lançar as tarefas pelo Rei_Subdito

Para ativar as tarefas pela GUI do REI_SUBDITO, procedemos da mesma forma para todas as Tarefas. Ao ativar a *checkbox* que diz respeito à Tarefa, esta é lançada. O tratamento dos eventos foi o seguinte:

```
checkBoxRei.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        controlRei();
    }
});

(...)

public void controlRei() {
    if (checkBoxRei.isSelected()) {
        rei = new Rei(bCircular);
        logTextArea.append("Rei iniciado \n");
        rei.start();
        rei.getHaTrabalho().release();
    }else {
        rei.setGuiClose(true);
        rei.getGui_REI().dispose();
        rei.interrupt();
        logTextArea.append("Rei fechado \n");
    }
}
```

```
checkBoxSubdito.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        controlSubdito();
    }
});

public void controlSubdito() {
    if (checkBoxSubdito.isSelected()) {
        subdito = new Subdito(bCircular);
        logTextArea.append("Subdito iniciado \n");
        robot = new MyRobotLegoEV3();
        subdito.setRobot(robot);
        subdito.start();
    }else {
        subdito.setGuiClose(true);
        subdito.getGui_SUB().dispose();
        subdito.interrupt();
        subdito = null;
        logTextArea.append("Subdito fechado \n");
    }
}
```

```

checkBoxGravar.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        controlGravar();
    }
});

public void controlGravar() {
    if(subdito != null) {
        if (checkBoxGravar.isSelected()) {
            gravar_Spy = new Gravar_Spy();
            new Thread(gravar_Spy).start();
            gravar_Spy.setMyRobotLegoEV3(subdito.getRobot());
            subdito.setRobot(gravar_Spy);
            logTextArea.append("Gravar iniciado \n");
        }else {
            robot.setMyRobotLegoEV3(subdito.getRobot());
            subdito.setRobot(robot);
            gravar_Spy.getGravarGUI().dispose();
            new Thread(gravar_Spy).interrupt();
            logTextArea.append("Gravar fechado \n");
        }
    } else if(!checkBoxGravar.isSelected()) {
        gravar_Spy.getGravarGUI().dispose();
        new Thread(gravar_Spy).interrupt();
        logTextArea.append("Gravar fechado \n");
    }else {
        logTextArea.append("Checkbox do Subdito nao esta selecionada \n");
        checkBoxGravar.setSelected(false);
    }
}
}

```

O Gravar tem a particularidade de colocar-se com o papel de *robot* do Subdito. Além disso, a Tarefa Gravar também se distingue por ser inicializada de forma diferente das outras, por implementar a Interface Runnable.

Com esta implementação já temos as tarefas a serem lançadas pelo Rei_Subdito. Se desmarcados a *checkbox* e voltarmos a marcá-la, é lançada uma nova Tarefa.

Aula 6 - Teste da aplicação multitarefa completa.

1. 1 - Teste da aplicação

A comunicação entre o Rei e o Súbdito não apresentou nenhum problema, assim como a comunicação entre o Gravar e o Súbito e o lançamento das Tarefas através do Rei_Subdito. Contudo, ao reproduzir o que se tinha gravado no ficheiro o resultado não foi o esperado.

O motivo para isso foi o grupo não saber como calcular o tempo de execução do primeiro comando gravado. O tempo de execução era calculado da seguinte forma:

Ao clicar num botão de comando, como “Frente”, o *timestamp* inicial era registado e quando se voltava a clicar noutro botão de comando, esse instante servia como *timestamp* final do primeiro comando e como *timestamp* inicial do próximo comando.

Verificou-se, através de alguns testes que envolveram *prints* e colocar um tempo *defalut* para o primeiro comando, que o tempo de execução era respeitado para os restantes comandos. Contudo, o grupo não conseguiu contornar este obstáculo.

Conclusão

Este trabalho serviu para conhecer novos métodos e classes Java tais como os que foram usados na escrita e leitura de ficheiro, no Gravar. Também aprendemos sobre semáforos e monitores e como implementar os mesmos.

O grupo teve algumas dificuldades no final do trabalho com a temporização dos comandos para reproduzi-los e também reconheceu que a gestão dos semáforos que controla as Tarefas, poderia estar implementada de forma mais otimizada e eficiente.

Bibliografia

Consulta:

Mooddle:

Pais, Jorge. (2023 - 2024). *Fundamentos de Sistemas Operativos*

Java ObjectInputStream (With Examples). (n.d.). [Www.programiz.com](http://www.programiz.com).

<https://www.programiz.com/java-programming/objectinputstream>

Paleta de cores:

Color Palette: #27374D #526D82 #9DB2BF #DDE6ED - Color Hunt. (n.d.). Colorhunt.co.

<https://colorhunt.co/palette/27374d526d829db2bfdde6ed>


```

private JCheckBox checkBoxSubdito;
private JCheckBox checkBoxGravar;
private JButton btnNewSubdito;
private JLabel labelLog;
private JButton buttonCleanLog;
private JScrollPane scrollPane;
private JTextArea logTextArea;
private boolean selectRei;
private boolean selectSubdito;
private boolean selectGravar;

private BufferCircular bCircular;

private Rei rei;
private Subdito subdito;
private Gravar_Spy gravar_Spy;
private MyRobotLegoEV3 robot;

public static void main(String[] args) {
    GUI_REI_SUBDITO frame = new GUI_REI_SUBDITO();
}

public GUI_REI_SUBDITO() {

    this.selectRei = false;
    this.selectSubdito = false;
    this.selectGravar = false;
    bCircular = new BufferCircular(12);

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 450, 380);
    contentPane = new JPanel();
    contentPane.setBackground(new Color(38, 55, 77));
    contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));

    setContentPane(contentPane);
    this.setTitle("TP2 - REI SUBDITO");

    panel = new JPanel();
    panel.setForeground(new Color(221, 230, 237));
    panel.setBackground(new Color(38, 55, 77));
    panel.setBorder(new TitledBorder(new EtchedBorder(EtchedBorder.LOWERED, new Color(255, 255, 255), new Color(160, 160, 160)), "Ativar/Desativar Tarefa", TitledBorder.LEADING, TitledBorder.TOP, null, new Color(221, 230, 237)));
    panel.setBounds(10, 10, 416, 65);
    panel.setLayout(null);

    checkBoxRei = new JCheckBox("REI");
    checkBoxRei.setForeground(new Color(221, 230, 237));
    checkBoxRei.setBackground(new Color(38, 55, 77));
    checkBoxRei.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent arg0) {
            controlRei();
        }
    });
    checkBoxRei.setFont(new Font("Tahoma", Font.PLAIN, 14));
    checkBoxRei.setBounds(6, 24, 78, 21);
    panel.add(checkBoxRei);

    checkBoxSubdito = new JCheckBox("SUBDITO");
    checkBoxSubdito.setForeground(new Color(221, 230, 237));
    checkBoxSubdito.setBackground(new Color(38, 55, 77));
    checkBoxSubdito.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent arg0) {
            controlSubdito();
        }
    });
    checkBoxSubdito.setFont(new Font("Tahoma", Font.PLAIN, 14));
    checkBoxSubdito.setBounds(149, 24, 118, 21);
    panel.add(checkBoxSubdito);

    btnNewSubdito = new JButton("+");
    btnNewSubdito.setBounds(108, 24, 41, 21);
    panel.add(btnNewSubdito);
    btnNewSubdito.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent arg0) {
            addSubditos();
        }
    });
}

```

```

checkBoxGravar = new JCheckBox("GRAVAR");
checkBoxGravar.setForeground(new Color(221, 230, 237));
checkBoxGravar.setBackground(new Color(38, 55, 77));
checkBoxGravar.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        controlGravar();
    }
});
checkBoxGravar.setFont(new Font("Tahoma", Font.PLAIN, 14));
checkBoxGravar.setBounds(314, 24, 96, 21);
panel.add(checkBoxGravar);

panel_log = new JPanel();
panel_log.setBackground(new Color(38, 55, 77));
panel_log.setBounds(10, 96, 416, 237);
contentPane.add(panel_log);
panel_log.setLayout(null);

labelLog = new JLabel("Log");
labelLog.setForeground(new Color(221, 230, 237));
labelLog.setBackground(new Color(0, 64, 128));
labelLog.setFont(new Font("Tahoma", Font.PLAIN, 14));
labelLog.setBounds(10, 10, 48, 27);
panel_log.add(labelLog);

buttonCleanLog = new JButton("Limpar Log");
buttonCleanLog.setForeground(new Color(39, 55, 77));
buttonCleanLog.setBackground(new Color(221, 230, 237));
buttonCleanLog.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        logTextArea.setText(null);
    }
});
buttonCleanLog.setFont(new Font("Tahoma", Font.PLAIN, 14));
buttonCleanLog.setBounds(303, 10, 103, 27);
panel_log.add(buttonCleanLog);

contentPane.setLayout(null);
contentPane.add(panel);
contentPane.add(panel_log);

scrollPane = new JScrollPane();
scrollPane.setBounds(0, 47, 416, 180);
panel_log.add(scrollPane);

logTextArea = new JTextArea();
logTextArea.setForeground(new Color(221, 230, 237));
logTextArea.setBackground(new Color(82, 109, 130));
logTextArea.setFont(new Font("Monospaced", Font.PLAIN, 14));
scrollPane.setViewportView(logTextArea);

setVisible(true);
}

public void controlRei() {
    if (checkBoxRei.isSelected()) {
        rei = new Rei(bCircular);
        logTextArea.append("Rei iniciado \n");
        rei.start();
        rei.getHaTrabalho().release();
    } else {
        rei.setGuiClose(true);
        rei.getGui_REI().dispose();
        rei.interrupt();
        logTextArea.append("Rei fechado \n");
    }
}

public void controlSubdito() {
    if (checkBoxSubdito.isSelected()) {
        subdito = new Subdito(bCircular);
        logTextArea.append("Subdito iniciado \n");
        robot = new MyRobotLegoEV3();
        subdito.setRobot(robot);
        subdito.start();
    } else {
        subdito.setGuiClose(true);
        subdito.getGui_SUB().dispose();
    }
}

```

```

        subdito.interrupt();
        subdito = null;
        logTextArea.append("Subdito fechado \n");
    }
}

public void controlGravar() {
    if(subdito != null) {
        if (checkBoxGravar.isSelected()) {
            gravar_Spy = new Gravar_Spy();
            new Thread(gravar_Spy).start();
            gravar_Spy.setMyRobotLegoEV3(subdito.getRobot());
            subdito.setRobot(gravar_Spy);
            logTextArea.append("Gravar iniciado \n");
        } else {
            robot.setMyRobotLegoEV3(subdito.getRobot());
            subdito.setRobot(robot);
            gravar_Spy.getGravarGUI().dispose();
            new Thread(gravar_Spy).interrupt();
            logTextArea.append("Gravar fechado \n");
        }
    } else if(!checkBoxGravar.isSelected()) {
        gravar_Spy.getGravarGUI().dispose();
        new Thread(gravar_Spy).interrupt();
        logTextArea.append("Gravar fechado \n");
    } else {
        logTextArea.append("Checkbox do Subdito nao esta selecionada \n");
        checkBoxGravar.setSelected(false);
    }
}

public void addSubditos() {
    if (checkBoxSubdito.isSelected()) {
        subdito = new Subdito(bCircular);
        logTextArea.append("Novo Subdito iniciado \n");
        robot = new MyRobotLegoEV3();
        subdito.setRobot(robot);
        subdito.start();
    } else {
        logTextArea.append("Checkbox nao esta selecionada \n");
    }
}
}
}

```

Código GUI Gravar:

*//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
 //Fundamentos de Sistemas Operativos - TP2 - Aula 6
 //Coding UTF-08*

// GUI Gravar

```

import java.awt.EventQueue;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.border.EmptyBorder;
import javax.swing.border.TitledBorder;
import javax.swing.JTextField;
import javax.swing.JLabel;
import java.awt.Font;
import javax.swing.JButton;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JCheckBox;
import javax.swing.JFileChooser;

import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
import java.awt.Color;
import javax.swing.border.EtchedBorder;

public class GRAVAR extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JPanel panel_File;
    private JPanel panel_Gravar_Reproduzir;

```

```

private JPanel log_panel;
private JTextField textField_filepath;
private JLabel label_file;
private JButton searchButton;
private JCheckBox checkBoxGravar_Parar;
private JCheckBox checkBoxReproduzir;
private JScrollPane scrollPane;
private JLabel label_log;
private JButton buttonCleanLog;
private JTextArea LogtextArea;
private boolean gravar_parar;
private boolean reproduzir;
private String filePath;

public static void main(String[] args) {
    GRAVAR gravar = new GRAVAR();
}

public GRAVAR() {
    setTitle("Trabalho 2 - Gravar");

    this.gravar_parar = false;
    this.reproduzir = false;

    setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    setBounds(100, 100, 450, 380);
    contentPane = new JPanel();
    contentPane.setBackground(new Color(39, 55, 77));
    contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));

    setContentPane(contentPane);
    contentPane.setLayout(null);

    panel_File = new JPanel();
    panel_File.setForeground(new Color(221, 230, 237));
    panel_File.setBackground(new Color(39, 55, 77));
    panel_File.setLayout(null);
    panel_File.setBorder(new TitledBorder(new EtchedBorder(EtchedBorder.LOWERED, new
Color(255, 255, 255), new Color(160, 160, 160)), "Definir o nome do ficheiro",
TitledBorder.LEADING, TitledBorder.TOP, null, new Color(221, 230, 237)));
    panel_File.setBounds(10, 0, 416, 69);

    textField_filepath = new JTextField();
    textField_filepath.setForeground(new Color(221, 230, 237));
    textField_filepath.setBackground(new Color(82, 109, 130));
    textField_filepath.setFont(new Font("Tahoma", Font.PLAIN, 12));
    textField_filepath.setColumns(10);
    textField_filepath.setBounds(67, 24, 276, 30);
    panel_File.add(textField_filepath);

    label_file = new JLabel("Ficheiro:");
    label_file.setForeground(new Color(221, 230, 237));
    label_file.setFont(new Font("Tahoma", Font.PLAIN, 14));
    label_file.setBounds(10, 30, 57, 13);
    panel_File.add(label_file);

    searchButton = new JButton("...");
    searchButton.setBackground(new Color(221, 230, 237));
    searchButton.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            JFileChooser fileChooser = new JFileChooser();
            int result = fileChooser.showOpenDialog(null);

            if (result == JFileChooser.APPROVE_OPTION) {
                setFilePath(fileChooser.getSelectedFile().getAbsolutePath());
                textField_filepath.setText(filePath);
            }
        }
    });
    searchButton.setFont(new Font("Tahoma", Font.PLAIN, 14));
    searchButton.setBounds(353, 24, 43, 30);
    panel_File.add(searchButton);

    panel_Gravar_Reproduzir = new JPanel();
    panel_Gravar_Reproduzir.setBackground(new Color(39, 55, 77));
    panel_Gravar_Reproduzir.setLayout(null);
    panel_Gravar_Reproduzir.setBorder(new TitledBorder(new EtchedBorder(EtchedBorder.LOWERED,
new Color(255, 255, 255), new Color(160, 160, 160)), "Gravar & Reproduzir", TitledBorder.LEADING,

```

```

TitledBorder.TOP, null, new Color(221, 230, 237)));
panel_Gravar_Reproduzir.setBounds(10, 79, 416, 57);

checkBoxGravar_Parar = new JCheckBox("Gravar/Parar");
checkBoxGravar_Parar.setForeground(new Color(221, 230, 237));
checkBoxGravar_Parar.setBackground(new Color(39, 55, 77));
checkBoxGravar_Parar.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        setReproduzir(false);
        setGravar_parar(true);
        checkBoxReproduzir.setSelected(false);
    }
});
checkBoxGravar_Parar.setFont(new Font("Tahoma", Font.PLAIN, 14));
checkBoxGravar_Parar.setBounds(32, 16, 103, 25);
panel_Gravar_Reproduzir.add(checkBoxGravar_Parar);

checkBoxReproduzir = new JCheckBox("Reproduzir");
checkBoxReproduzir.setForeground(new Color(221, 230, 237));
checkBoxReproduzir.setBackground(new Color(39, 55, 77));
checkBoxReproduzir.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        setGravar_parar(false);
        setReproduzir(true);
        checkBoxGravar_Parar.setSelected(false);
    }
});
checkBoxReproduzir.setFont(new Font("Tahoma", Font.PLAIN, 14));
checkBoxReproduzir.setBounds(288, 16, 93, 25);
panel_Gravar_Reproduzir.add(checkBoxReproduzir);

log_panel = new JPanel();
log_panel.setBackground(new Color(39, 55, 77));
log_panel.setLayout(null);
log_panel.setBounds(10, 140, 416, 193);

scrollPane = new JScrollPane();
scrollPane.setBounds(0, 54, 416, 129);
log_panel.add(scrollPane);

label_log = new JLabel("Log");
label_log.setForeground(new Color(221, 230, 237));
label_log.setFont(new Font("Tahoma", Font.PLAIN, 14));
label_log.setBounds(11, 13, 54, 18);
log_panel.add(label_log);

buttonCleanLog = new JButton("Limpar Log");
buttonCleanLog.setBackground(new Color(221, 230, 237));
buttonCleanLog.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        LogTextArea.setText(null);
    }
});
buttonCleanLog.setFont(new Font("Tahoma", Font.PLAIN, 13));
buttonCleanLog.setBounds(294, 10, 110, 34);
log_panel.add(buttonCleanLog);

contentPane.add(panel_File);
contentPane.add(panel_Gravar_Reproduzir);
contentPane.add(log_panel);

LogTextArea = new JTextArea();
LogTextArea.setForeground(new Color(221, 230, 237));
LogTextArea.setFont(new Font("Tahoma", Font.PLAIN, 18));
LogTextArea.setBackground(new Color(82, 109, 130));
scrollPane.setViewportView(LogTextArea);

setVisible(true);
}

public boolean isGravar_parar() {
    return gravar_parar;
}

public void setGravar_parar(boolean gravar_parar) {
    this.gravar_parar = gravar_parar;
}

public boolean isReproduzir() {
    return reproduzir;
}

```

```

    }

    public void setReproduzir(boolean reproduzir) {
        this.reproduzir = reproduzir;
    }

    public String getFilePath() {
        return filePath;
    }

    public void setFilePath(String filePath) {
        this.filePath = filePath;
    }

    public JTextArea getLogtextArea() {
        return LogtextArea;
    }

    public void setLogtextArea(JTextArea logtextArea) {
        LogtextArea = logtextArea;
    }
}

```

Código Classe MyRobotLegoEV3:

*//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
 //Fundamentos de Sistemas Operativos - TP2 - Aula 6
 //Coding UTF-08*

//Classe MyRobotLegoEV3

```

import robot.RobotLegoEV3;

public class MyRobotLegoEV3{
    private String nomeRobot;
    private boolean onOff;
    private RobotLegoEV3 robot;

    public MyRobotLegoEV3() {
        nomeRobot = "EV4"; //Pass 1234
        onOff = false;
        robot = new RobotLegoEV3();
    }

    public void setMyRobotLegoEV3(MyRobotLegoEV3 r) {
        nomeRobot = r.getNomeRobot(); //Pass 1234
        onOff = r.isOnOff();
        robot = r.getRobot();
    }

    public void frente(int distancia) {
        robot.Reta(distancia);
    }

    public boolean Open(String nome) {
        return robot.OpenEV3(nome);
    }

    public void Close() {
        robot.CloseEV3();
    }

    public void tras(int distancia) {
        robot.Reta(-1*distancia);
    }

    public void curvaDireita(int raio, int angulo) {
        robot.CurvarDireita(raio, angulo);
    }

    public void curvaEsquerda(int raio, int angulo) {
        robot.CurvarEsquerda(raio, angulo);
    }

    public void parar(boolean parar) {
        robot.Parar(parar);
    }
}

```

```

    }

    public boolean isOnOff() {
        return onOff;
    }

    public void setOnOff(boolean onOff) {
        this.onOff = onOff;
    }

    public String getNomeRobot() {
        return nomeRobot;
    }

    public void setNomeRobot(String nomeRobot) {
        this.nomeRobot = nomeRobot;
    }

    public RobotLegoEV3 getRobot() {
        return robot;
    }

    public void setRobot(RobotLegoEV3 robot) {
        this.robot = robot;
    }
}

```

Código Classe Gravar_Spy:

```

//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
//Fundamentos de Sistemas Operativos - TP2 - Aula 6
//Coding UTF-08

//Gravar

import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.time.Duration;
import java.time.Instant;
import java.util.ArrayList;
import java.util.concurrent.Semaphore;

import org.xml.sax.ext.LexicalHandler;

import javafx.scene.shape.MeshView;
import robot.RobotLegoEV3;

public class Gravar_Spy extends MyRobotLegoEV3 implements IGravar_Spy, Runnable {

    private GRAVAR gravarGUI;
    private boolean guiClose;
    private int state;
    private Mensagem msg;
    private MensagemTempo msgReadCheck;
    private MensagemTempo msgGrava;

    private int contIdx;
    private boolean fiscalTrabalho;

    private long timeStamp;
    private long timeDuration;
    private long tempoMsgAnterior;

    private Semaphore haTrabalho;
    private Semaphore gravar;

    private File fileGravar;
    private FileInputStream inputStream;
    private FileOutputStream outputStream;
    private ObjectInputStream objectInput;
    private ObjectOutputStream objectOutput;

```

```

public Gravar_Spy() {

    setNomeRobot("Gravar_SpyRobot");

    gravarGUI = new GRAVAR();
    state = CRIAR_FILE;
    msg = new Mensagem();
    msgReadCheck = new MensagemTempo();
    msgGrava = new MensagemTempo();

    tempoMsgAnterior = -1;

    contIdx = 1;
    fiscalTrabalho = false;
    haTrabalho = new Semaphore(0);
    gravar = new Semaphore(0);

    gravarGUI.addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            setGuiClose(true); // A janela foi fechada
        }
    });
}

public void run() {
    while (state != END_STATE) {
        automato();
    }
}

public void automato() {
    switch (state) {
        case CRIAR_FILE:
            if (gravarGUI.getFilePath() != null) {
                fileGravar = new File(gravarGUI.getFilePath());
                try {
                    if (fileGravar.createNewFile()) {
                        outputStream = new FileOutputStream(fileGravar, true);
                        objectOutput = new ObjectOutputStream(outputStream);

                        inputStream = new FileInputStream(fileGravar);
                        objectInput = new ObjectInputStream(inputStream);

                        state = ESPERAR_ATIVIDADE;
                        break;
                    }
                } catch (IOException e) {
                    e.printStackTrace();
                }
            } else {
                try {
                    Thread.sleep(1);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
            break;
        case ESPERAR_TRABALHO:
            try {
                haTrabalho.acquire();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            state = ESPERAR_ATIVIDADE;
            break;
        case ESPERAR_ATIVIDADE:
            if (isGuiClose()) {
                state = END_STATE;
                break;
            }
            if (gravarGUI.isGravar_parar() && fiscalTrabalho) {
                state = GRAVAR_PARAR;
            } else if (gravarGUI.isReproduzir()) {
                try {
                    if (objectInput.available() > 0) {
                        contIdx = 1;
                        state = REPRODUZIR;
                    }
                } catch (IOException e) {

```

```

        e.printStackTrace();
    }
} else {
    state = ESPERAR_ATIVIDADE;
}
break;
case GRAVAR_PARAR:

    try {
        gravar.acquire();

        if (msgGrava != msg) {

            objectOutput.write(msgGrava.getNum());
            objectOutput.write(msgGrava.getCmd());
            objectOutput.write(msgGrava.getArg1());
            objectOutput.write(msgGrava.getArg2());
            objectOutput.writeLong(msgGrava.getTempo());

            objectOutput.flush();

            gravarGUI.getLogTextArea().append("G:" + msgGrava.toString() + "\n");
            msg.setNum(msgGrava.getNum());
            msg.setCmd(msgGrava.getCmd());
            msg.setArg1(msgGrava.getArg1());
            msg.setArg2(msgGrava.getArg2());

        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    setFiscalTrabalho(false);
    haTrabalho.release();
    state = ESPERAR_TRABALHO;
    break;
case REPRODUZIR:

    try {
        msgReadCheck.setNum(objectInput.read());
        msgReadCheck.setCmd(objectInput.read());
        msgReadCheck.setArg1(objectInput.read());
        msgReadCheck.setArg2(objectInput.read());
        msgReadCheck.setTempo(objectInput.readLong());

        if (tempoMsgAnterior == -1) {
            timeDuration = timeStamp - msgReadCheck.getTempo();
        } else {
            timeDuration = msgReadCheck.getTempo() - tempoMsgAnterior;
        }

        tempoMsgAnterior = msgReadCheck.getTempo();

        gravarGUI.getLogTextArea().append("R:" + msgReadCheck.toString() + "\n");

        if (msgReadCheck.getCmd() == CMD_RETA) {
            frente(msgReadCheck.getArg1());
        }
        if (msgReadCheck.getCmd() == CMD_CDIREITA) {
            curvaDireita(msgReadCheck.getArg1(), msgReadCheck.getArg2());
        }
        if (msgReadCheck.getCmd() == CMD_CESQUERDA) {
            curvaEsquerda(msgReadCheck.getArg1(), msgReadCheck.getArg2());
        }
        if (msgReadCheck.getCmd() == CMD_PARAR) {
            parar(true);
        }

        try {
            Thread.sleep(timeDuration);
        } catch (Exception e) {
            System.out.println("Erro no sleep GRAVAR");
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
    haTrabalho.release();
    state = ESPERAR_TRABALHO;
    break;
}

```

```

    }

    public synchronized void frente(int distancia) {
        timeStamp = System.currentTimeMillis();

        super.frente(distancia);
        if (gravarGUI.isGravar_parar()) {
            msgGrava.setNum(contIdx);
            msgGrava.setCmd(CMD_RETA);
            msgGrava.setArg1(distancia);
            msgGrava.setArg2(0);
            msgGrava.setTempo(timeStamp);

            contIdx++;
            setFiscalTrabalho(true);
            gravar.release();
        }
    }

    public synchronized void tras(int distancia) {

        timeStamp = System.currentTimeMillis();

        super.tras(distancia);
        if (gravarGUI.isGravar_parar()) {
            msgGrava.setNum(contIdx);
            msgGrava.setCmd(CMD_RETA);
            msgGrava.setArg1(-1 * distancia);
            msgGrava.setArg2(0);
            msgGrava.setTempo(timeStamp);
            contIdx++;
            setFiscalTrabalho(true);
            gravar.release();
        }
    }

    public synchronized void curvaDireita(int raio, int angulo) {

        timeStamp = System.currentTimeMillis();

        super.curvaDireita(raio, angulo);
        if (gravarGUI.isGravar_parar()) {
            msgGrava.setNum(contIdx);
            msgGrava.setCmd(CMD_CDIREITA);
            msgGrava.setArg1(raio);
            msgGrava.setArg2(angulo);
            msgGrava.setTempo(timeStamp);
            contIdx++;
            setFiscalTrabalho(true);
            gravar.release();
        }
    }

    public synchronized void curvaEsquerda(int raio, int angulo) {

        timeStamp = System.currentTimeMillis();

        super.curvaEsquerda(raio, angulo);
        if (gravarGUI.isGravar_parar()) {
            msgGrava.setNum(contIdx);
            msgGrava.setCmd(CMD_CESQUERDA);
            msgGrava.setArg1(raio);
            msgGrava.setArg2(angulo);
            msgGrava.setTempo(timeStamp);
            contIdx++;
            setFiscalTrabalho(true);
            gravar.release();
        }
    }

    public synchronized void parar(boolean parar) {
        timeStamp = System.currentTimeMillis();

        super.parar(parar);
        if (gravarGUI.isGravar_parar()) {
            msgGrava.setNum(contIdx);
            msgGrava.setCmd(CMD_PARAR);
            msgGrava.setArg1(1);
            msgGrava.setArg2(0);

```

```

        msgGrava.setTempo(timeStamp);

        contIdx++;
        setFiscalTrabalho(true);
        gravar.release();
    }

}

public GRAVAR getGravarGUI() {
    return gravarGUI;
}

public void setGravarGUI(GRAVAR gravarGUI) {
    this.gravarGUI = gravarGUI;
}

public boolean isGuiClose() {
    return guiClose;
}

public void setGuiClose(boolean guiClose) {
    this.guiClose = guiClose;
}

public int getState1() {
    return state;
}

public void setState1(int state) {
    this.state = state;
}

public Semaphore getHaTrabalho() {
    return haTrabalho;
}

public void setHaTrabalho(Semaphore haTrabalho) {
    this.haTrabalho = haTrabalho;
}

public File getGravaFile() {
    return fileGravar;
}

public void setGravaFile(File gravaFile) {
    this.fileGravar = gravaFile;
}

public FileInputStream getInputStream() {
    return inputStream;
}

public void setInputStream(FileInputStream inputStream) {
    this.inputStream = inputStream;
}

public FileOutputStream getOutputStream() {
    return outputStream;
}

public void setOutputStream(FileOutputStream outputStream) {
    this.outputStream = outputStream;
}

public ObjectInputStream getObjectInput() {
    return objectInput;
}

public void setObjectInput(ObjectInputStream objectInput) {
    this.objectInput = objectInput;
}

public ObjectOutputStream getObjectOutput() {
    return objectOutput;
}

public void setObjectOutput(ObjectOutputStream objectOutput) {
    this.objectOutput = objectOutput;
}

```

```

    }

    public Mensagem getMsg() {
        return msg;
    }

    public void setMsg(Mensagem msg) {
        this.msg = msg;
    }

    public Mensagem getMsgReadCheck() {
        return msgReadCheck;
    }

    public void setMsgReadCheck(MensagemTempo msgReadCheck) {
        this.msgReadCheck = msgReadCheck;
    }

    public MensagemTempo getMsgGrava() {
        return msgGrava;
    }

    public void setMsgGrava(MensagemTempo msgGrava) {
        this.msgGrava = msgGrava;
    }

    public boolean isFiscalTrabalho() {
        return fiscalTrabalho;
    }

    public void setFiscalTrabalho(boolean fiscalTrabalho) {
        this.fiscalTrabalho = fiscalTrabalho;
    }
}

```

Código Classe Gui Genérica:

*//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
 //Fundamentos de Sistemas Operativos - TP2 - Aula 6
 //Coding UTF-08*

```

//Classe Gui Genérica

import java.awt.EventQueue;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.border.EmptyBorder;
import javax.swing.JLabel;
import java.awt.Font;
import javax.swing.JTextField;
import javax.swing.JButton;
import javax.swing.LayoutStyle.ComponentPlacement;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
import javax.swing.border.TitledBorder;
import javax.swing.JSpinner;
import javax.swing.JRadioButton;
import javax.swing.SpinnerNumberModel;
import javax.swing.border.EtchedBorder;
import java.awt.Color;
import javax.swing.JCheckBox;
import javax.swing.JFileChooser;
import javax.swing.JTextArea;
import javax.swing.JScrollPane;
import javax.swing.JScrollBar;

public class Generico extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JTextField RaiotextField;
    private JTextField AngulotextField;
    private JTextField DisttextField;
    private JPanel controlPanel;
    private JLabel RaioLabel;
    private JButton btnRetaguarda;

```

```

private JLabel anguloLabel;
private JPanel LogPanel;
private JCheckBox ativarDesativarCheckBox;
private JLabel logLabel;
private JTextArea LogtextArea;
private JButton LimparLog;
private JLabel distLabel;
private JButton DireitaButton;
private JButton PararButton;
private JButton EsquerdaButton;
private JButton FrenteButton;

//minhas variáveis

Dados dados;
private JScrollPane scrollPane;

public void iniVariaveis() {
    dados = new Dados();
}

public boolean getAtivar() {
    return ativarDesativarCheckBox.isSelected();
}
/**
 * Create the frame.
 */
public Generico() {

    //instancia das minhas variáveis

    iniVariaveis();

    setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    setBounds(100, 100, 650, 521);
    contentPane = new JPanel();
    contentPane.setBackground(new Color(39, 55, 77));
    contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));

    setContentPane(contentPane);
    controlPanel = new JPanel();
    controlPanel.setForeground(new Color(221, 230, 237));
    controlPanel.setBackground(new Color(39, 55, 77));
    controlPanel.setBounds(10, 10, 600, 139);
    controlPanel.setLayout(null);
    controlPanel.setBorder(new TitledBorder(new EtchedBorder(EtchedBorder.LOWERED, new
Color(255, 255, 255), new Color(160, 160, 160)), "Controle do Robot", TitledBorder.LEADING,
TitledBorder.TOP, null, new Color(221, 230, 237)));

    RaioLabel = new JLabel("Raio");
    RaioLabel.setForeground(new Color(221, 230, 237));
    RaioLabel.setFont(new Font("Tahoma", Font.PLAIN, 14));
    RaioLabel.setBounds(41, 29, 54, 20);
    controlPanel.add(RaioLabel);

    RaiotextField = new JTextField("" + dados.getRaio());
    RaiotextField.setBackground(new Color(82, 109, 130));
    RaiotextField.setForeground(new Color(221, 230, 237));
    RaiotextField.setFont(new Font("Tahoma", Font.PLAIN, 14));
    RaiotextField.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent arg0) {
            try {
                dados.setRaio(Integer.parseInt(RaiotextField.getText()));
            } catch (NumberFormatException nfe) {
                RaiotextField.setText("" + dados.getRaio());
            }
        }
    });
    RaiotextField.setColumns(10);
    RaiotextField.setBounds(122, 31, 113, 19);
    controlPanel.add(RaiotextField);

    btnRetaguarda = new JButton("Retaguarda");
    btnRetaguarda.setForeground(new Color(39, 55, 77));
    btnRetaguarda.setBackground(new Color(221, 230, 237));
    btnRetaguarda.setFont(new Font("Tahoma", Font.PLAIN, 14));
    btnRetaguarda.setBounds(386, 101, 109, 29);
    controlPanel.add(btnRetaguarda);

```

```

anguloLabel = new JLabel("Ângulo");
anguloLabel.setForeground(new Color(221, 230, 237));
anguloLabel.setFont(new Font("Tahoma", Font.PLAIN, 14));
anguloLabel.setBounds(41, 59, 54, 20);

controlPanel.add(anguloLabel);
LogPanel = new JPanel();
LogPanel.setBackground(new Color(39, 55, 77));
LogPanel.setBounds(20, 235, 590, 247);
LogPanel.setLayout(null);

ativarDesativarCheckBox = new JCheckBox("Ativar/Desativar Comportamento");
ativarDesativarCheckBox.setForeground(new Color(221, 230, 237));
ativarDesativarCheckBox.setBackground(new Color(39, 55, 77));
ativarDesativarCheckBox.setFont(new Font("Tahoma", Font.PLAIN, 14));
ativarDesativarCheckBox.setBounds(19, 14, 251, 22);

LogPanel.add(ativarDesativarCheckBox);
logLabel = new JLabel("Log");
logLabel.setForeground(new Color(221, 230, 237));
logLabel.setFont(new Font("Tahoma", Font.PLAIN, 14));
logLabel.setBounds(19, 42, 54, 20);
LogPanel.add(logLabel);

LimparLog = new JButton("Limpar Log");
LimparLog.setForeground(new Color(39, 55, 77));
LimparLog.setBackground(new Color(221, 230, 237));
LimparLog.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        LogTextArea.setText(null);
    }
});
LimparLog.setFont(new Font("Tahoma", Font.PLAIN, 14));
LimparLog.setBounds(225, 211, 141, 33);
LogPanel.add(LimparLog);

AngulotextField = new JTextField("" + dados.getAngulo());
AngulotextField.setBackground(new Color(82, 109, 130));
AngulotextField.setForeground(new Color(221, 230, 237));
AngulotextField.setFont(new Font("Tahoma", Font.PLAIN, 14));
AngulotextField.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try {
            dados.setAngulo(Integer.parseInt(AngulotextField.getText()));
        } catch (NumberFormatException nfe) {
            AngulotextField.setText("" + dados.getAngulo());
        }
    }
});
AngulotextField.setColumns(10);
AngulotextField.setBounds(122, 60, 113, 19);
controlPanel.add(AngulotextField);

DisttextField = new JTextField("" + dados.getDistancia());
DisttextField.setBackground(new Color(82, 109, 130));
DisttextField.setForeground(new Color(221, 230, 237));
DisttextField.setFont(new Font("Tahoma", Font.PLAIN, 14));
DisttextField.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try {
            dados.setDistancia(Integer.parseInt(DisttextField.getText()));
        } catch (NumberFormatException nfe) {
            DisttextField.setText("" + dados.getDistancia());
        }
    }
});
DisttextField.setColumns(10);
DisttextField.setBounds(122, 96, 113, 19);
controlPanel.add(DisttextField);

distLabel = new JLabel("Distância");
distLabel.setForeground(new Color(221, 230, 237));
distLabel.setFont(new Font("Tahoma", Font.PLAIN, 14));
distLabel.setBounds(41, 94, 80, 20);
controlPanel.add(distLabel);

DireitaButton = new JButton("Direita");
DireitaButton.setForeground(new Color(39, 55, 77));
DireitaButton.setBackground(new Color(221, 230, 237));

```

```

DireitaButton.setFont(new Font("Tahoma", Font.PLAIN, 14));
DireitaButton.setBounds(499, 62, 90, 29);
controlPanel.add(DireitaButton);

PararButton = new JButton("Parar");
PararButton.setForeground(new Color(39, 55, 77));
PararButton.setBackground(new Color(221, 230, 237));
PararButton.setFont(new Font("Tahoma", Font.PLAIN, 14));
PararButton.setBounds(386, 62, 103, 29);
controlPanel.add(PararButton);

EsquerdaButton = new JButton("Esquerda");
EsquerdaButton.setForeground(new Color(39, 55, 77));
EsquerdaButton.setBackground(new Color(221, 230, 237));
EsquerdaButton.setFont(new Font("Tahoma", Font.PLAIN, 14));
EsquerdaButton.setBounds(278, 61, 97, 29);
controlPanel.add(EsquerdaButton);

FrenteButton = new JButton("Frente");
FrenteButton.setForeground(new Color(39, 55, 77));
FrenteButton.setBackground(new Color(221, 230, 237));
FrenteButton.setFont(new Font("Tahoma", Font.PLAIN, 14));
FrenteButton.setBounds(387, 26, 99, 29);
controlPanel.add(FrenteButton);

contentPane.setLayout(null);
contentPane.add(controlPanel);
contentPane.add(LogPanel);

scrollPane = new JScrollPane();
scrollPane.setBounds(19, 72, 561, 127);
LogPanel.add(scrollPane);

LogtextArea = new JTextArea();
LogtextArea.setForeground(new Color(221, 230, 237));
LogtextArea.setBackground(new Color(82, 109, 130));
scrollPane.setViewportView(LogtextArea);

setVisible(true);
}

public JButton getEsquerdaButton() {
    return EsquerdaButton;
}

public void setEsquerdaButton(JButton esquerdaButton) {
    EsquerdaButton = esquerdaButton;
}

public JButton getPararButton() {
    return PararButton;
}

public void setPararButton(JButton pararButton) {
    PararButton = pararButton;
}

public JButton getBtnRetaguarda() {
    return btnRetaguarda;
}

public void setBtnRetaguarda(JButton btnRetaguarda) {
    this.btnRetaguarda = btnRetaguarda;
}

public JButton getFrenteButton() {
    return FrenteButton;
}

public void setFrenteButton(JButton frenteButton) {
    FrenteButton = frenteButton;
}

public JButton getDireitaButton() {
    return DireitaButton;
}

public void setDireitaButton(JButton direitaButton) {
    DireitaButton = direitaButton;
}

public JTextArea getLogtextArea() {
    return LogtextArea;
}

public void setLogtextArea(JTextArea logtextArea) {
    LogtextArea = logtextArea;
}

public Dados getDados() {

```

```

        return dados;
    }
    public void setDados(Dados dados) {
        this.dados = dados;
    }
}

```

Código Classe Gui Rei:

```

//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
//Fundamentos de Sistemas Operativos - TP2 - Aula 6
//Coding UTF-08

//GUI Rei

import java.awt.EventQueue;
import javax.swing.JPanel;
import javax.swing.border.TitledBorder;
import javax.swing.JButton;
import javax.swing.LayoutStyle.ComponentPlacement;
import java.awt.Font;
import javax.swing.JScrollBar;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
import java.awt.Color;
import javax.swing.border.EtchedBorder;

public class GUI_REI extends Generico{

    private JPanel Controle_Panel;
    private JButton OitoCmdButton;
    private JButton DezasseisCmdButton;

    private Rei rei;

    public GUI_REI(Rei rei) {
        super();
        getContentPane().setForeground(new Color(221, 230, 237));

        this.rei = rei;

        this.setTitle("Trabalho 2 - Rei");

        Controle_Panel = new JPanel();
        Controle_Panel.setForeground(new Color(221, 230, 237));
        Controle_Panel.setBackground(new Color(39, 55, 77));
        Controle_Panel.setBorder(new TitledBorder(new EtchedBorder(EtchedBorder.LOWERED, new
Color(255, 255, 255), new Color(160, 160, 160)), "Controle do Robot em Modo Autom\u00E9tico",
TitledBorder.LEADING, TitledBorder.TOP, null, new Color(221, 230, 237)));
        Controle_Panel.setBounds(10, 163, 596, 62);

        DezasseisCmdButton = new JButton("16 Comandos Aleatórios");
        DezasseisCmdButton.setForeground(new Color(39, 55, 77));
        DezasseisCmdButton.setBackground(new Color(221, 230, 237));
        DezasseisCmdButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                rei.getPodeFuncionar().release();
                rei.setState2(IRei.SEND_16CMD);
            }
        });
        DezasseisCmdButton.setBounds(349, 15, 193, 34);
        DezasseisCmdButton.setFont(new Font("Tahoma", Font.PLAIN, 14));

        Controle_Panel.setLayout(null);
        Controle_Panel.add(DezasseisCmdButton);

        getContentPane().add(Controle_Panel);

        OitoCmdButton = new JButton("8 Comandos Aleatórios");
        OitoCmdButton.setForeground(new Color(39, 55, 77));
        OitoCmdButton.setBackground(new Color(221, 230, 237));
        OitoCmdButton.setBounds(39, 15, 193, 34);
        Controle_Panel.add(OitoCmdButton);
        OitoCmdButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent arg0) {
                rei.getPodeFuncionar().release();
            }
        });
    }
}

```

```

        rei.setState2(IRei.SEND_8CMD);
    }
});
OitoCmdButton.setFont(new Font("Tahoma", Font.PLAIN, 14));

getFrenteButton().addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        rei.getPodeFuncionar().release();
        rei.setState2(IRei.SEND_FRENTE);
    }
});

getBtnRetaguarda().addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        rei.getPodeFuncionar().release();
        rei.setState2(IRei.SEND_TRAS);
    }
});

getDireitaButton().addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        rei.getPodeFuncionar().release();
        rei.setState2(IRei.SEND_CDIREITA);
    }
});

getEsquerdaButton().addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        rei.getPodeFuncionar().release();
        rei.setState2(IRei.SEND_CESQUERDA);
    }
});

getPararButton().addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        rei.getPodeFuncionar().release();
        rei.setState2(IRei.SEND_PARAR);
    }
});
}
}

```

Código Classe Gui Súdito:

//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
 //Fundamentos de Sistemas Operativos - TP2 - Aula 6
 //Coding UTF-08

//Classe Gui Subdito

```

import javax.swing.JPanel;
import javax.swing.border.TitledBorder;
import javax.swing.JLabel;
import java.awt.Font;
import javax.swing.JTextField;
import javax.swing.JRadioButton;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
import java.awt.Color;
import javax.swing.border.EtchedBorder;
public class GUI_SUBDITO extends Generico {

    private JPanel Robot_panel;
    private JTextField nameTextField;
    private JLabel nomeRobotLabel;
    private JRadioButton abrirFecharRobotRadBtn;
    private boolean robotCheck;
    private boolean RobotRadioBtn;

    private Subdito subdito;

    public GUI_SUBDITO(Subdito subdito) {
        super();

        this.subdito = subdito;

        RobotRadioBtn = false;
    }
}

```

```

this.setTitle("Trabalho 2 - Subdito");

Robot_panel = new JPanel();
Robot_panel.setForeground(new Color(221, 230, 237));
Robot_panel.setBackground(new Color(39, 55, 77));
Robot_panel.setBorder(new TitledBorder(new EtchedBorder(EtchedBorder.LOWERED, new
Color(255, 255, 255), new Color(160, 160, 160)), "Robot", TitledBorder.LEADING, TitledBorder.TOP,
null, new Color(221, 230, 237)));
Robot_panel.setBounds(10, 162, 598, 69);
Robot_panel.setLayout(null);

nomeRobotLabel = new JLabel("Nome do Robot");
nomeRobotLabel.setForeground(new Color(221, 230, 237));
nomeRobotLabel.setFont(new Font("Tahoma", Font.PLAIN, 14));
nomeRobotLabel.setBounds(10, 25, 111, 13);
Robot_panel.add(nomeRobotLabel);

//nameTextField = new JTextField(dados.getNomeRobot());
nameTextField = new JTextField();
nameTextField.setForeground(new Color(221, 230, 237));
nameTextField.setBackground(new Color(82, 109, 130));
nameTextField.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        subdito.getRobot().setNomeRobot(nameTextField.getText());
    }
});
nameTextField.setBounds(118, 24, 261, 19);
Robot_panel.add(nameTextField);
nameTextField.setColumns(10);

abrirFecharRobotRadBtn = new JRadioButton("Abrir/Fechar Robot");
abrirFecharRobotRadBtn.setForeground(new Color(221, 230, 237));
abrirFecharRobotRadBtn.setBackground(new Color(39, 55, 77));
abrirFecharRobotRadBtn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        setRobotRadioBtn(true);
        if(!subdito.getRobot().isOnOff()) {
            if(subdito.getRobot().Open(subdito.getRobot().getNomeRobot())) {
                abrirFecharRobotRadBtn.setSelected(true);
                subdito.getRobot().setOnOff(true);
                getLogTextArea().append("Ligado ao RobotLegoEV3:" +
subdito.getRobot().getNomeRobot()+"\n");
                nameTextField.setText(getRobot_panel().getName());
            }
            else {
                abrirFecharRobotRadBtn.setSelected(false);
                getLogTextArea().append("Insucesso ao ligar o RobotLegoEV3 \n");
            }
        }
        else {
            abrirFecharRobotRadBtn.setSelected(false);
            subdito.getRobot().getRobot().CloseEV3();
            subdito.getRobot().setOnOff(false);
            getLogTextArea().append("Desligado com sucesso \n");
        }
    }
});
abrirFecharRobotRadBtn.setFont(new Font("Tahoma", Font.PLAIN, 14));
abrirFecharRobotRadBtn.setBounds(409, 22, 143, 21);
Robot_panel.add(abrirFecharRobotRadBtn);

getContentPane().add(Robot_panel);

getFrenteButton().addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        subdito.getRobot().frente(dados.getDistancia());
        subdito.getRobot().parar(false);
        getLogTextArea().append("Reta (Sucesso)" + dados.getDistancia() + "\n");
    }
});

getBtnRetaguarda().addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent arg0) {
        subdito.getRobot().tras(dados.getDistancia());
        subdito.getRobot().parar(false);
        getLogTextArea().append("Tras (Sucesso)" + -1*dados.getDistancia() + "\n");
    }
});

getDireitaButton().addActionListener(new ActionListener() {

```

```

        public void actionPerformed(ActionEvent arg0) {
            subdito.getRobot().curvaDireita(dados.getRaio(), dados.getAngulo());
            subdito.getRobot().parar(false);
            getLogTextArea().append("CDireira (Sucesso)" + dados.getRaio() + " " +
dados.getAngulo() + "\n");
        }
    });

    getEsquerdaButton().addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent arg0) {
            subdito.getRobot().curvaEsquerda(dados.getRaio(), dados.getAngulo());
            subdito.getRobot().parar(false);
            getLogTextArea().append("CEsquerda (Sucesso)" + dados.getRaio() + " " +
dados.getAngulo() + "\n");
        }
    });

    getPararButton().addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent arg0) {
            subdito.getRobot().parar(true);
            getLogTextArea().append("Parar (Sucesso) \n");
        }
    });
}

public JPanel getRobot_panel() {
    return Robot_panel;
}

public void setRobot_panel(JPanel robot_panel) {
    Robot_panel = robot_panel;
}

public JTextField getNameTextField() {
    return nameTextField;
}

public void setNameTextField(JTextField nameTextField) {
    this.nameTextField = nameTextField;
}

public JLabel getNomeRobotLabel() {
    return nomeRobotLabel;
}

public void setNomeRobotLabel(JLabel nomeRobotLabel) {
    this.nomeRobotLabel = nomeRobotLabel;
}

public boolean getAbrirFecharRobotRadBtn() {
    return abrirFecharRobotRadBtn.isSelected();
}

public void setAbrirFecharRobotRadBtn(Boolean valor) {
    abrirFecharRobotRadBtn.setSelected(valor);
}

public boolean isRobotRadioBtn() {
    return RobotRadioBtn;
}

public void setRobotRadioBtn(boolean RobotRadioBtn) {
    this.RobotRadioBtn = RobotRadioBtn;
}
}

```

Código Classe Rei:

```

//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
//Fundamentos de Sistemas Operativos - TP2 - Aula 6
//Coding UTF-08

//Classe Rei

import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.time.Instant;

```

```

import java.util.Arrays;
import java.util.concurrent.Semaphore;

public class Rei extends Thread implements IRei {
    private GUI_REI gui_REI;
    private int state;
    private int state2;
    private CriarMensagem cMsg;
    private Mensagem mensagem;
    private boolean guiClose;
    private int tempWait;
    private int cont16;
    private int cont8;

    // VARIÁVEIS NOVAS TP2
    private Semaphore haTrabalho;
    private Semaphore podeFuncionar;

    private BufferCircular bufferCircular;

    public Rei(BufferCircular bc) {
        gui_REI = new GUI_REI(this);

        state = ESPERAR_TRABALHO;
        state2 = CRIAR_OBJETO_MENSAGEM;
        cMsg = null;
        mensagem = null;
        guiClose = false;
        tempWait = 0;
        cont16 = 0;
        cont8=0;

        bufferCircular = bc;

        haTrabalho = new Semaphore(0);
        podeFuncionar = new Semaphore(0);

        gui_REI.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                setGuiClose(true); // A janela foi fechada
            }
        });
    }

    public void run() {
        while(state2!=END_STATE) {
            automato();
        }
    }

    public void automato() {

        switch (state) {
            case ESPERAR_TRABALHO:
                try {
                    haTrabalho.acquire();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }

                state = FUNCIONAR;
                break;

            case FUNCIONAR:
                funcionar();
                break;
        }
    }

    public void funcionar() {

        switch (state2) {
            case CRIAR_OBJETO_MENSAGEM:
                cMsg = new CriarMensagem(gui_REI.dados, 12);

                state2 = ESPERAR_ATIVAR;
                break;
            case ESPERAR_ATIVAR:

```

```

        if (cont16!=0 && cont8 != 0) {
            try {
                podedFuncionar.acquire();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }

        if (gui_REI.getAtivar()) {
            if(cont8!=0) {
                state2 = SEND_8CMD;
            }else if (cont16!=0) {
                state2 = SEND_16CMD;
            }else {
                state2 = getState2();
                break;
            }
        }

        if (isGuiClose()) {
            state2 = END_STATE;
            break;
        }

        break;
    case SEND_FRENTE:
        mensagem = cMsg.MsgFrente();
        bufferCircular.escrever(mensagem);
        gui_REI.getLogtextArea().append(mensagem.toString() + "\n");

        state2 = ESPERAR_ATIVAR;
        break;

    case SEND_TRAS:
        mensagem = cMsg.MsgTras();
        bufferCircular.escrever(mensagem);
        gui_REI.getLogtextArea().append(mensagem.toString() + "\n");

        state2 = ESPERAR_ATIVAR;
        break;

    case SEND_CDIREITA:
        mensagem = cMsg.MsgCurDireita();
        bufferCircular.escrever(mensagem);
        gui_REI.getLogtextArea().append(mensagem.toString() + "\n");

        state2 = ESPERAR_ATIVAR;
        break;

    case SEND_CESQUERDA:
        mensagem = cMsg.MsgCurEsquerda();
        bufferCircular.escrever(mensagem);
        gui_REI.getLogtextArea().append(mensagem.toString() + "\n");

        state2 = ESPERAR_ATIVAR;
        break;

    case SEND_PARAR:
        mensagem = cMsg.MsgParar();
        bufferCircular.escrever(mensagem);
        gui_REI.getLogtextArea().append(mensagem.toString() + "\n");

        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            System.err.println("A espera foi interrompida");
            e.printStackTrace();
        }

        state2 = ESPERAR_ATIVAR;
        break;

    case SEND_8CMD:
        if(cont8!=NUM_8CMD) {
            mensagem = cMsg.MsgAleatorio();

            bufferCircular.escrever(mensagem);
            gui_REI.getLogtextArea().append(mensagem.toString() + "\n");
            cont8++;
        }
    }
}

```

```

        state2 = ESPERAR_ATIVAR;
        break;
    }else {
        cont8=0;
    }

    state2 = ESPERAR_ATIVAR;
    break;
case SEND 16CMD:
    if(cont16!=NUM_16CMD) {
        mensagem = cMsg.MsgAleatorio();

        bufferCircular.escrever(mensagem);
        gui_REI.getLogTextArea().append(mensagem.toString() + "\n");
        cont16++;

        state2 = ESPERAR_ATIVAR;
        break;
    }else {
        cont16=0;
    }

    state2 = ESPERAR_ATIVAR;
    break;
case END_STATE:
    try {
        Thread.sleep(100);
    } catch (InterruptedException e) {
        System.err.println("A espera foi interrompida");
        e.printStackTrace();
    }
    break;
}

}

public boolean isGuiClose() {
    return guiClose;
}

public void setGuiClose(boolean guiClose) {
    this.guiClose = guiClose;
}

public GUI_REI getGui_REI() {
    return gui_REI;
}

public void setGui_REI(GUI_REI gui_REI) {
    this.gui_REI = gui_REI;
}

public int getState1() {
    return state;
}

public void setState1(int state) {
    this.state = state;
}

public int getState2() {
    return state2;
}

public void setState2(int state2) {
    this.state2 = state2;
}

public Semaphore getHaTrabalho() {
    return haTrabalho;
}

public void setHaTrabalho(Semaphore haTrabalho) {
    this.haTrabalho = haTrabalho;
}

public Semaphore getPodeFuncionar() {
    return podeFuncionar;
}
}

```

```

    public void setPodeFuncionar(Semaphore podeFuncionar) {
        this.podeFuncionar = podeFuncionar;
    }

    /*public static void main(String[] args) {
        Rei r = new Rei();
        r.run();
    }*/
}

```

Código Classe Subdito:

```

//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
//Fundamentos de Sistemas Operativos - TP2 - Aula 6
//Coding UTF-08

```

```

//Classe Subdio

```

```

import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Iterator;
import java.util.concurrent.Semaphore;
import java.lang.Math;
import javax.accessibility.AccessibleTableModelChange;

public class Subdito extends Thread implements ISubdito {
    private GUI_SUBDITO gui_SUB;
    private int state;
    private int state2;
    private boolean guiClose;
    private Mensagem msg;
    private Mensagem msg_Recebe;
    private int idxUltMsg;
    private int tempWait;
    private int distancia_robot;
    private double angulo_rad;

    private MyRobotLegoEV3 robot;

    private Semaphore haTrabalho;
    private BufferCircular bufferCircular;

    private ArrayList<Mensagem> memoria;

    public Subdito(BufferCircular bc) {
        gui_SUB = new GUI_SUBDITO(this);
        state = ESPERAR_ATIVAR_COMPORTAMENTO;
        state2 = ATIVAR_ROBOT;
        msg = new Mensagem();
        msg_Recebe = null;
        guiClose = false;
        idxUltMsg = 0;
        tempWait = 0;
        distancia_robot=0;
        angulo_rad=0;

        robot = null;
        memoria = new ArrayList<Mensagem>();

        haTrabalho = new Semaphore(0);
        bufferCircular = bc;

        gui_SUB.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                setGuiClose(true); // A janela foi fechada
            }
        });
    }

    public void run() {
        while(state!=END_STATE) {

```

```

        automato_Comunicacao();
        exeComando_Robot();
    }
}

public void automato_Comunicacao() {
    switch (state) {
        case ESPERAR_TRABALHO:
            //System.out.println("ESPERAR TRABALHO");
            try {
                haTrabalho.acquire();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }

            state = ESPERAR_ATIVAR_COMPORTAMENTO;
            break;
        case ESPERAR_ATIVAR_COMPORTAMENTO:
            //System.out.println("ESPERAR ATIVAR");

            if (gui.SUB.getAtivar() &&
                bufferCircular.getElementosOcupados().availablePermits() != 0) {
                state = LER_BUFFER;
                break;
            }
            break;
        case LER_BUFFER:
            //System.out.println("READ_MSG_CC");
            if (isGuiClose()) {
                state = END_STATE;
                break;
            }

            //System.out.println(Arrays.toString(bufferCircular.getBuffer()));
            //System.out.println(idxUltMsg);
            msg = bufferCircular.ler();

            state = GUARDAR_MENSAGEM_MEMORIA;
            break;
        case GUARDAR_MENSAGEM_MEMORIA:
            //System.out.println("SAVE_MSG_MEM");
            memoria.add(new Mensagem(msg));
            //System.out.println("Mem : "+memoria);
            haTrabalho.release();
            state = ESPERAR_TRABALHO;
            break;
    }
}

public void exeComando_Robot() {
    switch (state2) {
        case ATIVAR_ROBOT:
            //System.out.println("ATIVAR ROBOT");
            if (gui.SUB.isRobotRadioBtn()) {
                state2 = CHECK_TAMANHO_MEMORIA;
                break;
            } else {
                try {
                    Thread.sleep(1);
                } catch (InterruptedException e) {
                    System.err.println("Erro no sleep");
                    e.printStackTrace();
                }
            }
            state2 = CHECK_TAMANHO_MEMORIA;
            break;
        case CHECK_TAMANHO_MEMORIA:
            //System.out.println("DORMIR");
            if (idxUltMsg != memoria.size()) {
                state2 = READ_MSG_MEM;
            }
            break;
        case READ_MSG_MEM:
            //System.out.println("READ_MSG_MEM");
            if (idxUltMsg == memoria.size()) {
                state2 = CHECK_TAMANHO_MEMORIA;
                break;
            }
            if (memoria.get(idxUltMsg).getCmd() == CMD_PARAR) {
                state2 = EXE_PARAR;
            }
    }
}

```

```

        break;
    }
    if (memoria.get(idxUltMsg).getCmd() == CMD_RETA) {
        state2 = EXE_RETA;
        break;
    }
    if (memoria.get(idxUltMsg).getCmd() == CMD_CurDireita) {
        state2 = EXE_CurDireita;
        break;
    }
    if (memoria.get(idxUltMsg).getCmd() == CMD_CurEsquerda) {
        state2 = EXE_CurEsquerda;
        break;
    }
}
case EXE_RETA:
    getRobot().frente(memoria.get(idxUltMsg).getArg1());

    gui_SUB.getLogtextArea().append(memoria.get(idxUltMsg).toString() + " \n");

    distancia_robot = memoria.get(idxUltMsg).getArg1();

    state2 = WAIT_STATE;
    break;
case EXE_CurDireita:
    getRobot().curvaDireita(memoria.get(idxUltMsg).getArg1(),
memoria.get(idxUltMsg).getArg2());
    gui_SUB.getLogtextArea().append(memoria.get(idxUltMsg).toString() + " \n");

    angulo_rad = memoria.get(idxUltMsg).getArg2() * (2 * (int) Math.PI) / 360;
    distancia_robot = (int) angulo_rad * memoria.get(idxUltMsg).getArg1();

    state2 = WAIT_STATE;
    break;
case EXE_CurEsquerda:
    getRobot().curvaEsquerda(memoria.get(idxUltMsg).getArg1(),
memoria.get(idxUltMsg).getArg2());
    gui_SUB.getLogtextArea().append(memoria.get(idxUltMsg).toString() + " \n");
    angulo_rad = memoria.get(idxUltMsg).getArg2() * (2 * (int) Math.PI) / 360;
    distancia_robot = (int) angulo_rad * memoria.get(idxUltMsg).getArg1();

    state2 = WAIT_STATE;
    break;
case EXE_PARAR:
    getRobot().parar(true);
    distancia_robot = 0;
    gui_SUB.getLogtextArea().append(memoria.get(idxUltMsg).toString() + " \n");
    state2 = WAIT_STATE;
    break;
case WAIT_STATE:
    if (distancia_robot != 0) {
        tempWait = distancia_robot / VEL;
        try {
            Thread.sleep(tempWait * 1000 + TEMPO_COMUNIC);
        } catch (Exception e) {
            System.out.println("Erro no sleep");
        }
    } else {
        try {
            Thread.sleep(TEMPO_COMUNIC);
        } catch (Exception e) {
            System.out.println("Erro no sleep");
        }
    }
    idxUltMsg++;

    state2 = CHECK_TAMANHO_MEMORIA;
    break;
}
}

/*public static void main(String[] args) {
    Subdito s = new Subdito();
    s.run();
}*/

public GUI_SUBDITO getGui_SUB() {
    return gui_SUB;
}

public void setGui_SUB(GUI_SUBDITO gui_SUB) {

```

```

        this.gui_SUB = gui_SUB;
    }

    public int getStatel() {
        return state;
    }

    public void setStatel(int statel) {
        this.state = statel;
    }

    public boolean isGuiClose() {
        return guiClose;
    }

    public void setGuiClose(boolean guiClose) {
        this.guiClose = guiClose;
    }

    public Mensagem getMensagem() {
        return msg;
    }

    public void setMensagem(Mensagem msg) {
        this.msg = msg;
    }

    public ArrayList<Mensagem> getMemoria() {
        return memoria;
    }

    public void setMemoria(ArrayList<Mensagem> memoria) {
        this.memoria = memoria;
    }

    public Semaphore getHaTrabalho() {
        return haTrabalho;
    }

    public void setHaTrabalho(Semaphore haTrabalho) {
        this.haTrabalho = haTrabalho;
    }

    public MyRobotLegoEV3 getRobot() {
        return robot;
    }

    public void setRobot(MyRobotLegoEV3 robot) {
        this.robot = robot;
    }
}

```

Código Classe BufferCircular:

*//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
 //Fundamentos de Sistemas Operativos - TP2 - Aula 6
 //Coding UTF-08*

//Classe BufferCircular

```

import java.io.IOException;
import java.util.concurrent.Semaphore;

public class BufferCircular {
    private Mensagem[] buffer;
    private int put;
    private int get;
    private int size;

    private Semaphore elementosLivres;
    private Semaphore elementosOcupados;
    private Semaphore acessoElementos;

    public BufferCircular(int dim) {
        size = dim;
        buffer = new Mensagem[size];
        get = 0;
    }
}

```

```

        put = 0;
        elementosLivres = new Semaphore(size);
        elementosOcupados = new Semaphore(0);
        acessoElementos = new Semaphore(1);
    }

    public void escrever(Mensagem m) {
        try {
            elementosLivres.acquire();
            acessoElementos.acquire();
            buffer[put] = new Mensagem(m);
            put = ++put % size;
        } catch (InterruptedException e) {
        } finally {
            acessoElementos.release();
            elementosOcupados.release();
        }
    }

    public Mensagem ler() {
        Mensagem myMsg = null;
        try {
            elementosOcupados.acquire();
            acessoElementos.acquire();

        } catch (InterruptedException e) {
        } finally {
            myMsg = new Mensagem(buffer[get]);
            get = ++get % size;
            acessoElementos.release();
            elementosLivres.release();
        }
        return myMsg;
    }

    public int getSize() {
        return size;
    }

    public void setSize(int size) {
        this.size = size;
    }

    public int getPut() {
        return put;
    }

    public void setPut(int put) {
        this.put = put;
    }

    public int getGet() {
        return get;
    }

    public void setGet(int get) {
        this.get = get;
    }

    public Mensagem[] getBuffer() {
        return buffer;
    }

    public void setBuffer(Mensagem[] buffer) {
        this.buffer = buffer;
    }

    public Semaphore getElementosLivres() {
        return elementosLivres;
    }

    public void setElementosLivres(Semaphore elementosLivres) {
        this.elementosLivres = elementosLivres;
    }

    public Semaphore getElementosOcupados() {
        return elementosOcupados;
    }
}

```

```

    public void setElementosOcupados(Semaphore elementosOcupados) {
        this.elementosOcupados = elementosOcupados;
    }

    public Semaphore getAcessoElementos() {
        return acessoElementos;
    }

    public void setAcessoElementos(Semaphore acessoElementos) {
        this.acessoElementos = acessoElementos;
    }
}

```

Código Classe Mensagem:

```

//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
//Fundamentos de Sistemas Operativos - TP2 - Aula 6
//Coding UTF-08

//Classe Mensagem

import java.io.Serializable;

public class Mensagem implements IMensagem, Serializable{

    private int num;
    private int cmd;
    private int arg1;
    private int arg2;

    // Construtor
    public Mensagem() {
        this.num = 0;
        this.cmd = 9;
        this.arg1 = 0;
        this.arg2 = 0;
    }

    //CONSTRUTOR PARA COLOCAR AS MENSAGEM
    //NO ARRAYLIST (MEMÓRIA) DA FORMA PRETENDIDA
    public Mensagem(Mensagem arrayMensagem) {
        this.num = arrayMensagem.num;
        this.cmd = arrayMensagem.cmd;
        this.arg1 = arrayMensagem.arg1;
        this.arg2 = arrayMensagem.arg2;
    }

    public int getArg1() {
        return arg1;
    }

    public void setArg1(int arg1) {
        this.arg1 = arg1;
    }

    public int getArg2() {
        return arg2;
    }

    public void setArg2(int arg2) {
        this.arg2 = arg2;
    }

    public int getNum() {
        return num;
    }

    public void setNum(int num) {
        this.num = num;
    }

    public int getCmd() {
        return cmd;
    }
}

```

```

    public void setCmd(int cmd) {
        this.cmd = cmd;
    }

    public void print() {
        System.out.println(this.num + " " + this.cmd + " " + this.arg1 + " " + this.arg2);
    }

    public String toString() {
        return "Msg: num: " + this.num + " cmd: " + this.cmd + " arg1: " + this.arg1 + " arg2: " +
this.arg2;
    }
}

```

Código Classe MensagemTempo:

```

//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
//Fundamentos de Sistemas Operativos - TP2 - Aula 6
//Coding UTF-08

//Classe MensagemTempo

public class MensagemTempo extends Mensagem{

    private long tempo;

    public MensagemTempo() {
        super();
        this.tempo = 0;
    }

    public long getTempo() {
        return tempo;
    }

    public void setTempo(long tempo) {
        this.tempo = tempo;
    }

    public void print() {
        System.out.println(this.getNum() + " " + this.getCmd() + " " + this.getArg1() + " " +
this.getArg2() + " " + this.tempo);
    }

    public String toString() {
        return "Msg: num: " + this.getNum() + " cmd: " + this.getCmd() + " arg1: " + this.getArg1()
+ " arg2: " + this.getArg2()+ " tempo: " + this.tempo;
    }
}

```

Código Classe CriarMensagem:

```

//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
//Fundamentos de Sistemas Operativos - TP2 - Aula 6
//Coding UTF-08

//Classe CriarMensagem

import java.util.Random;

public class CriarMensagem {

    private Dados d;
    private Mensagem msg;
    private int numUltMsg;
    private Random random;
    private int bufferDim;

    public CriarMensagem(Dados dados, int bufferDim) {
        d = dados;
        msg = null;
        this.numUltMsg = 0;
    }
}

```

```

        bufferDim= bufferDim;
    }

    public Mensagem MsgFrente() {
        msg = new Mensagem();
        this.numUltMsg++;
        msg.setNum(this.numUltMsg);
        msg.setCmd(msg.CmdReta);
        msg.setArg1(d.getDistancia());
        msg.setArg2(0);
        return msg;
    }

    public Mensagem MsgTras() {
        msg = new Mensagem();
        this.numUltMsg++;
        msg.setNum(this.numUltMsg);
        msg.setCmd(msg.CmdReta);
        msg.setArg1(-1*d.getDistancia());
        msg.setArg2(0);
        return msg;
    }

    public Mensagem MsgParar() {
        msg = new Mensagem();
        this.numUltMsg++;
        msg.setNum(this.numUltMsg);
        msg.setCmd(msg.CmdParar);
        msg.setArg1(1);
        msg.setArg2(0);
        return msg;
    }

    public Mensagem MsgCurDireita() {
        msg = new Mensagem();
        this.numUltMsg++;
        msg.setNum(this.numUltMsg);
        msg.setCmd(msg.CmdCurvaDireita);
        msg.setArg1(d.getRaio());
        msg.setArg2(d.getAngulo());
        return msg;
    }

    public Mensagem MsgCurEsquerda() {
        msg = new Mensagem();
        this.numUltMsg++;
        msg.setNum(this.numUltMsg);
        msg.setCmd(msg.CmdCurvaEsquerda);
        msg.setArg1(d.getRaio());
        msg.setArg2(d.getAngulo());
        return msg;
    }

    public Mensagem MsgAleatorio() {
        random = new Random();
        msg = new Mensagem();
        this.numUltMsg++;
        msg.setNum(this.numUltMsg);
        msg.setCmd((int) (Math.random() * 3) + 1);
        if (msg.getCmd()==msg.CmdReta) {
            msg.setArg1((int) (Math.random()* 40) + 10 );
            msg.setArg2(0);
        }else {
            msg.setArg1((int) (Math.random() * 30) + 1);
            msg.setArg2((int) (Math.random() * 360));
        }

        return msg;
    }

    public int getBufferDim() {
        return bufferDim;
    }

    public void setBufferDim(int bufferDim) {
        this.bufferDim= bufferDim;
    }

```

```

    public Dados getD() {
        return d;
    }

    public void setD(Dados d) {
        this.d = d;
    }

    public Mensagem getMsg() {
        return msg;
    }

    public void setMsg(Mensagem msg) {
        this.msg = msg;
    }

    public int getNumUltMsg() {
        return numUltMsg;
    }

    public void setNumUltMsg(int numUltMsg) {
        this.numUltMsg = numUltMsg;
    }

    public void minusNum() {
        this.numUltMsg = this.numUltMsg-1;
    }
}

```

Código Classe Dados:

*//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
 //Fundamentos de Sistemas Operativos - TP2 - Aula 6
 //Coding UTF-08*

//Classe Dados

```

public class Dados {

    private int raio, angulo, distancia;

    //Construtor por defeito

    public Dados() {
        raio = 10;
        angulo = 90;
        distancia = 20;
    }

    public int getRaio() {
        return raio;
    }

    public void setRaio(int raio) {
        this.raio = raio;
    }

    public int getAngulo() {
        return angulo;
    }

    public void setAngulo(int angulo) {
        this.angulo = angulo;
    }

    public int getDistancia() {
        return distancia;
    }

    public void setDistancia(int distancia) {
        this.distancia = distancia;
    }
}

```

Código Interface Gravar_Spy:

```
//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
//Fundamentos de Sistemas Operativos - TP2 - Aula 6
//Coding UTF-08

//Interface IGravar_Spy

public interface IGravar_Spy {
    final int CRIAR_FILE = 0;
    final int INATIVO = 0;
    final int ESPERAR_TRABALHO = 1;
    final int ESPERAR_ATIVIDADE = 2;
    final int GRAVAR_PARAR = 3;
    final int REPRODUZIR = 4;
    final int END_STATE = 5;

    final int CMD_PARAR = 0;
    final int CMD_RETA = 1;
    final int CMD_CDIREITA = 2;
    final int CMD_CESQUERDA = 3;
}
```

Código Interface Rei:

```
//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
//Fundamentos de Sistemas Operativos - TP2 - Aula 6
//Coding UTF-08

//Interface Rei

public interface IRei {
    final int FUNCIONAR = 0;
    final int SELECT_MSG = 1;
    final int SEND_FRENTE = 2;
    final int SEND_TRAS = 3;
    final int SEND_CDIREITA = 4;
    final int SEND_CESQUERDA = 5;
    final int SEND_8CMD = 6;
    final int SEND_16CMD = 7;
    final int SEND_PARAR = 8;
    final int END_STATE = 9;
    final int ESPERAR_TRABALHO = 10;
    final int ABRIR_CANAL = 11;
    final int ESPERAR_ATIVAR = 12;
    final int CRIAR_OBJETO_MENSAGEM = 13;
    final int NUM_8CMD = 8;
    final int NUM_16CMD = 16;
    final int VEL = 20;
    final int TEMPO_COMUNIC = 100;
}
```

Código Interface Subdito:

```
//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
//Fundamentos de Sistemas Operativos - TP2 - Aula 6
//Coding UTF-08

//Interface Subdito

public interface ISubdito {
    final int ESPERAR_ATIVAR_COMPORTAMENTO = 0;
    final int ATIVAR_ROBOT = 12;
    final int ABRIR_CANAL = 2;
    final int LER_BUFFER = 3;
    final int GUARDAR_MENSAGEM_MEMORIA = 4;
    final int CHECK_TAMANHO_MEMORIA = 5;
    final int EXE_PARAR = 6;
    final int EXE_RETA = 7;
    final int EXE_CurDireita = 8;
    final int EXE_CurEsquerda = 9;
    final int READ_MSG_MEM = 10;
    final int VEL = 20;
    final int TEMPO_COMUNIC = 100;
    final int END_STATE = 11;
    final int WAIT_STATE = 13;
    final int FUNCIONAR=14;
    final int ESPERAR_TRABALHO = 15;
}
```

```
final int CMD_PARAR = 0;
final int CMD_RETA = 1;
final int CMD_CurDireita = 2;
final int CMD_CurEsquerda = 3;
}
```

Código Interface Mensagem:

```
//ISEL- LEIM - Miguel Alcobia, Fábio Pestana, João Ramos
//Fundamentos de Sistemas Operativos - TP2 - Aula 6
//Coding UTF-08

//Interface Mensagem

public interface IMensagem {
    final int CmdParar = 0;
    final int CmdReta = 1;
    final int CmdCurvaDireita = 2;
    final int CmdCurvaEsquerda = 3;
    final int CmdMsgLida = 4;
    final int CmdMsgVazia = 5;
}
```